KU LEUVEN



Decision trees

What are decison trees?



KU LEUVEN

What is a decision tree?

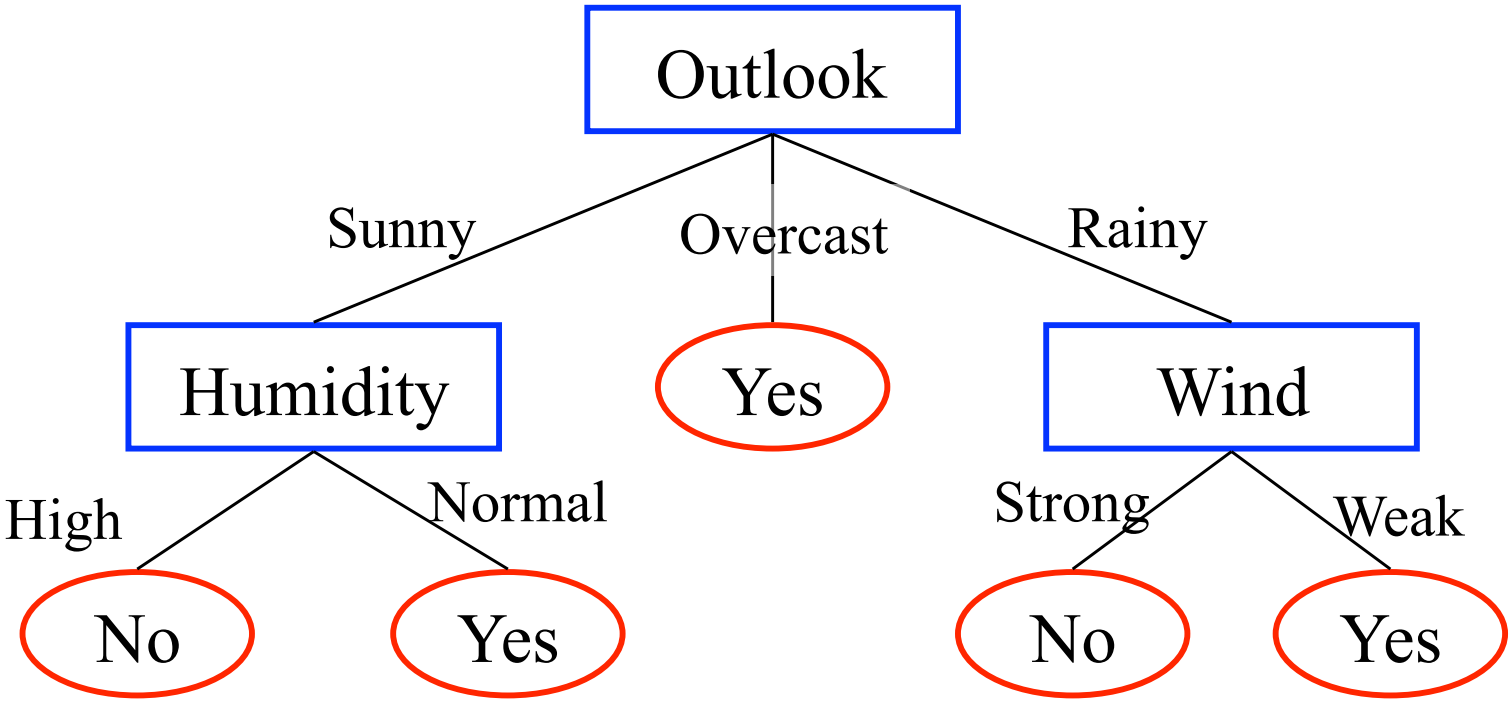
- A decision tree represents a decision procedure where
  - you start with one question
  - the answer to that question determines which question you should ask next
  - and so on, until you reach a decision

- We will usually call the questions “tests”, and the decision a “prediction”

Example decision tree

Do I want to play tennis today, or not?

The tree below represents one possible decision procedure (which takes the weather conditions into account).



What does the tree represent?

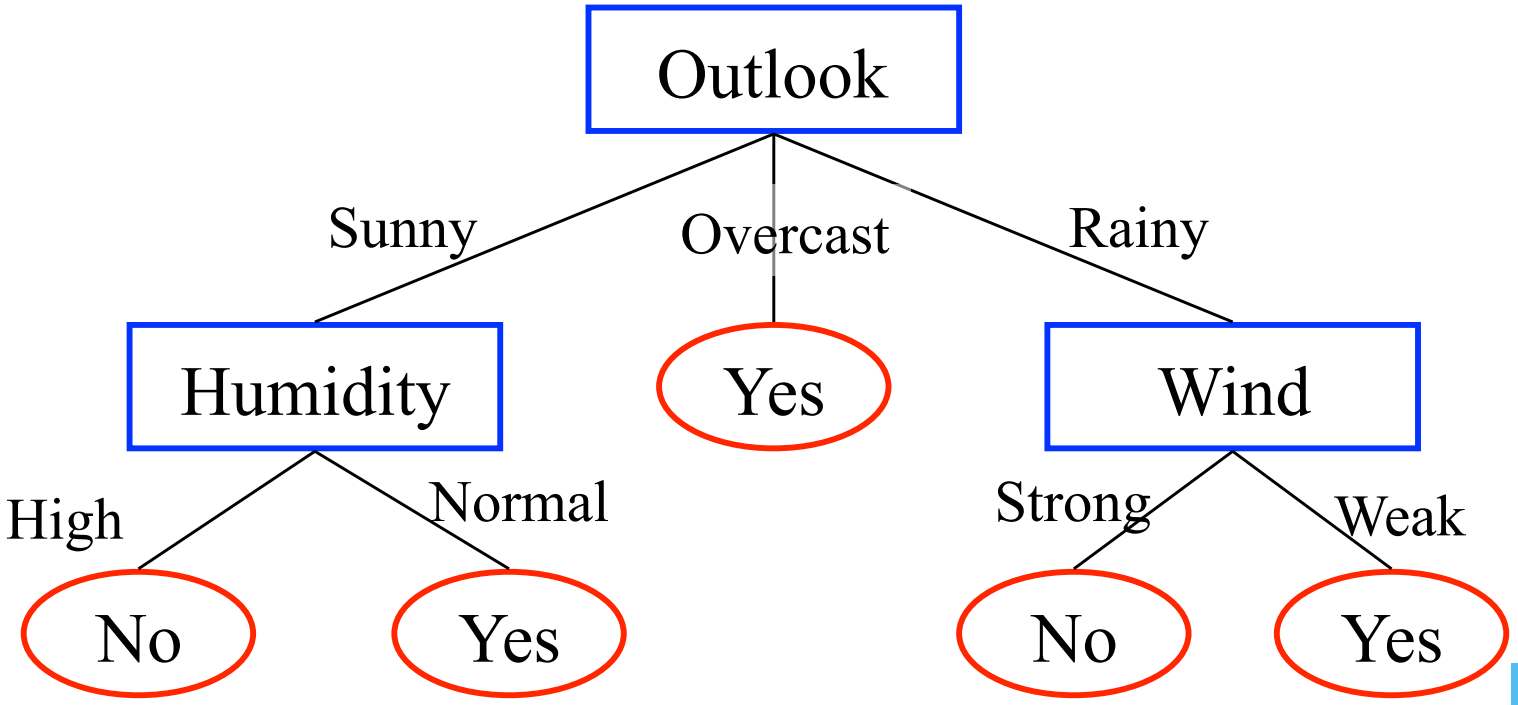
- Assume given a set of input attributes $X_1, X_2, \ldots, X_n$ and a target attribute Y
  - The questions can involve any X_i
  - The decision assigns a value to Y
- We will denote the input space by $\boldsymbol{X}$; points in $\boldsymbol{X}$ are vectors $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$
- Any $\boldsymbol{x}$ is mapped to exactly one value y for Y
- Hence, the tree represents a function from $\boldsymbol{X}$ to Y

KU LEUVEN

Illustration

Here, X_1 = *Outlook*, with possible values {Sunny, Overcast, Rainy}; X_2 = *Humidity* (High, Normal), X_3 = *Wind* (Strong, Weak), Y = *Tennis* (Yes, No)

The tree represents a function *Outlook* × *Humidity* × *Wind* → *Tennis*



Representation power

- Assume that the input attributes $X_1, X_2, \ldots, X_n$ and the output attribute Y are boolean (true/false)
- Trees then represent boolean functions
- **Can every imaginable boolean function be represented?**

- Try to write down a tree for:
  - $\vee$, the OR function : $X_1 \vee X_2$ = true if X_1, or X_2, or both are true
  - $\wedge$, the AND function: $X_1 \wedge X_2$ = true if X_1 and X_2 are both true
  - $\oplus$, the XOR function: $X_1 \oplus X_2$ = true if exactly one of X_1 and X_2 is true
  - $(A \wedge B) \vee (C \wedge \neg D \wedge E)$ (¬ means NOT: ¬D = true if D = false)

KU LEUVEN

Continuous input attributes

- What if one of the input attributes is a continuous attribute? E.g., X_1 is a number between 0 and 1
- We cannot make a different child node for each possible value!
- Tests must have a finite number of possible outcomes
- Solution: use comparative tests, e.g., $X_1 < 0.5$ (two possible outcomes: yes, no)

KU LEUVEN

Types of trees

- When Y is nominal, the tree is called a **classification tree**
- When Y is numerical, the tree is called a **regression tree**
- (There are other types of trees: see later)

KU LEUVEN

Why trees?

- If we want to learn a predictive function $f : X \rightarrow Y$, why would we want to learn it in the form of a decision tree?
  - there are many alternatives: neural network, support vector machine, probabilistic model, …
- Main reasons (as we will see later on):
  - learning and using trees is very **efficient**
  - they tend to have **good predictive accuracy**, even better when combined into ensembles (see elsewhere)
  - they are **interpretable**: we can understand what the tree says, and we can explain its predictions

How can we learn decision trees from data?

The basic learning algorithm



KU LEUVEN

Learning decision trees

- A decision tree represents a function $T : \boldsymbol{X} \rightarrow Y$
- Predictive learning is about learning such functions from data

- We assume given:
  - existence of some unknown function $f : \boldsymbol{X} \rightarrow Y$
  - a dataset D containing "examples" $(\boldsymbol{x}, f(\boldsymbol{x}))$ with $\boldsymbol{x} \in \boldsymbol{X}$
- We want to find:
  - a tree $T : \boldsymbol{X} \rightarrow Y$ that fulfills some criterion C (C depends on the task)

KU LEUVEN

Learning decision trees: task 1

Find the smallest tree T such that $\forall(\boldsymbol{x}, f(\boldsymbol{x})) \in D : T(\boldsymbol{x}) = f(\boldsymbol{x})$

(= smallest tree *consistent* with the data)

- Useful: uncovers the link between $\boldsymbol{X}$ and Y in the data set
  - tells us under what conditions Y takes certain values
  - this provides insight in the data
  - smallest tree = simplest explanation
- Mostly of theoretical interest
  - Often, D is only a sample from some larger distribution $\mathscr{D}$, and we want T to be accurate over $\mathscr{D}$, not only D

KU LEUVEN

Learning decision trees: task 2

Find the tree T such that for $\boldsymbol{x}$ drawn from $\mathscr{D}$,

$T(\boldsymbol{x})$ is (on average) maximally similar to $f(\boldsymbol{x})$

- This is formally expressed using a loss function $\ell : Y \times Y \rightarrow \mathbb{R}$
  - $\ell(y_1, y_2)$ indicates how bad it is to predict y_1 when you should have predicted y_2 (the higher the number, the worse)
- The **risk** R of T, relative to f, is $\mathbf{E}_{\boldsymbol{x} \sim \mathscr{D}}[(\ell(T(\boldsymbol{x}), f(\boldsymbol{x})]$
  (the expected value of $\ell(T(\boldsymbol{x}), f(\boldsymbol{x}))$, with $\boldsymbol{x}$ drawn from $\mathscr{D}$)
- Task, restated: find the tree with minimal risk

Learning decision trees: task 2

- In practice, we usually do not know $\mathscr{D}$, so we cannot compute the risk of a tree T, let alone find the tree with minimal risk

- Practical algorithms for learning decision trees will therefore use heuristics to find a tree that, hopefully,
  - is small (so it provides insight in the connection between $\boldsymbol{X}$ and Y)
  - generalizes well (so it can be used to make predictions outside D)

  using only the data in D

KU LEUVEN

Hardness of decision tree learning

- Known result from complexity theory: “learning decision trees is NP-hard”
- What is meant here, is that *finding the smallest tree that is consistent with the data is NP-hard*
- In practice, that is not what learners try to do
- We will see later on that practical algorithms for learning decision trees are in fact very efficient

KU LEUVEN

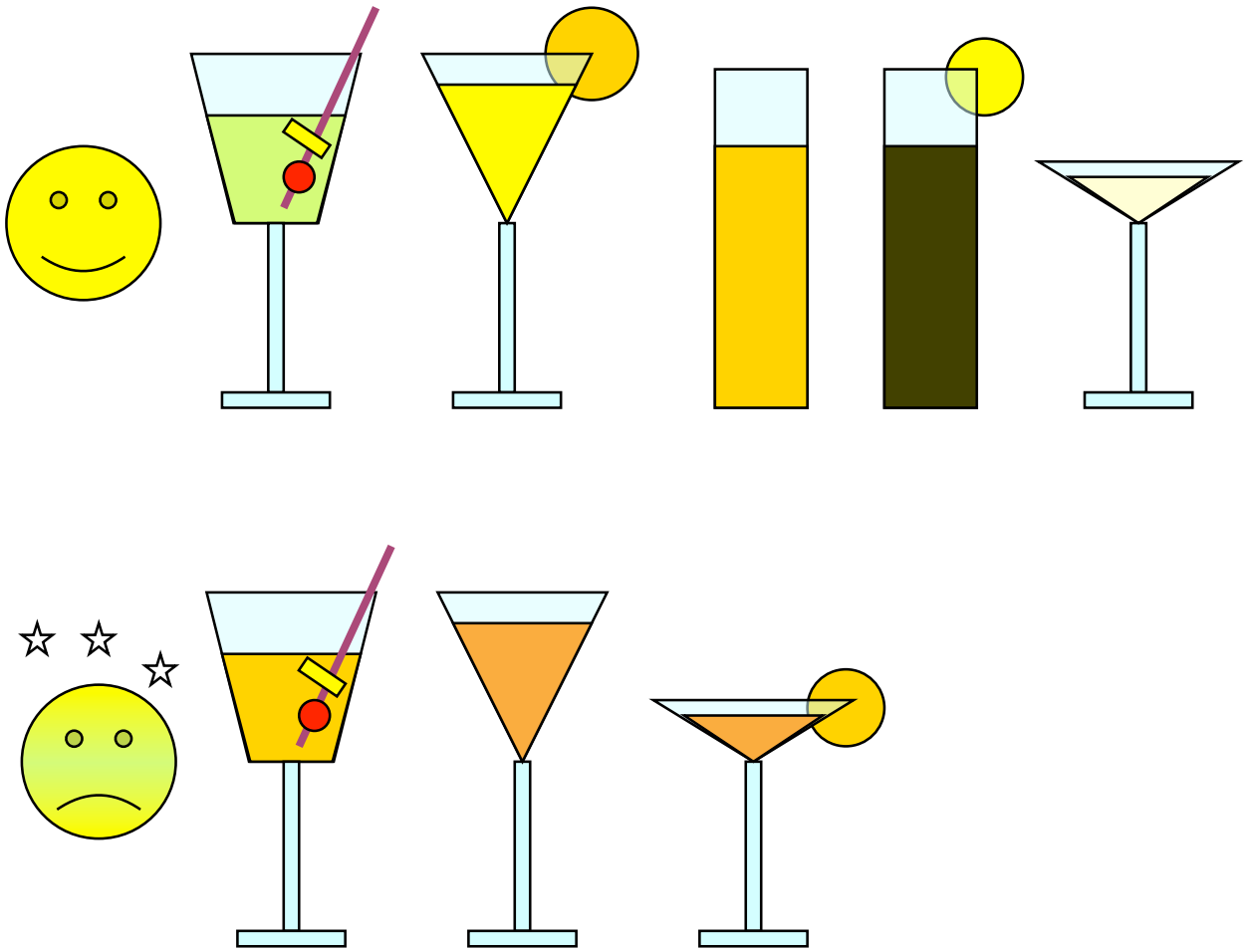
Learning DTs: the basic principle

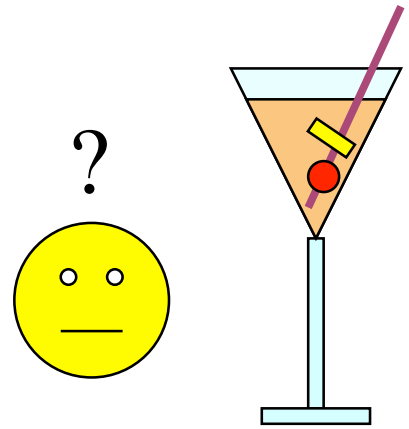
This approach is known as “Top-down induction of decision trees” (TDIDT), or as “recursive partitioning”

- Start with the full data set D
- Find a test such that examples in D with the same outcome for the test tend to have the same value of Y
- Split D into subsets, one for each outcome of that test
- Repeat this procedure on each subset that is not yet sufficiently “pure” (meaning, not all elements have the same Y)
- Keep repeating until no further splits possible

KU LEUVEN

Example: cocktail drinking

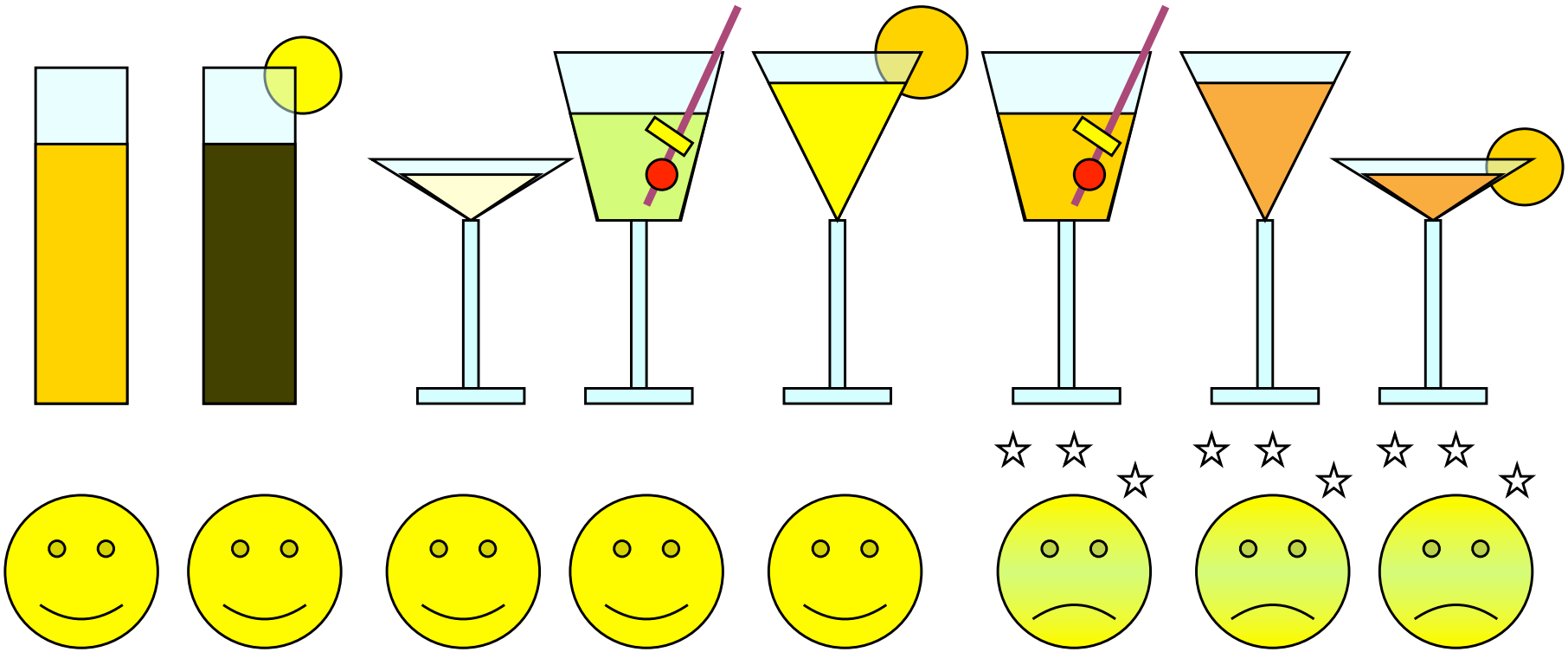




Will I get sick if I drink this, or not?

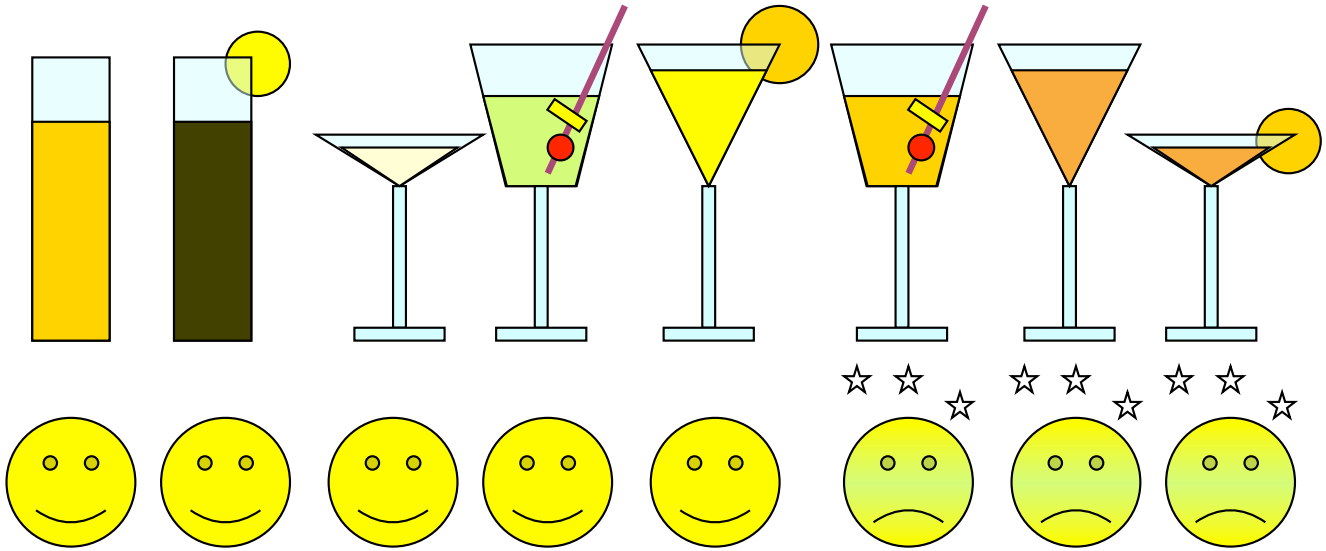
KU LEUVEN

The data set D: 8 examples



KU LEUVEN

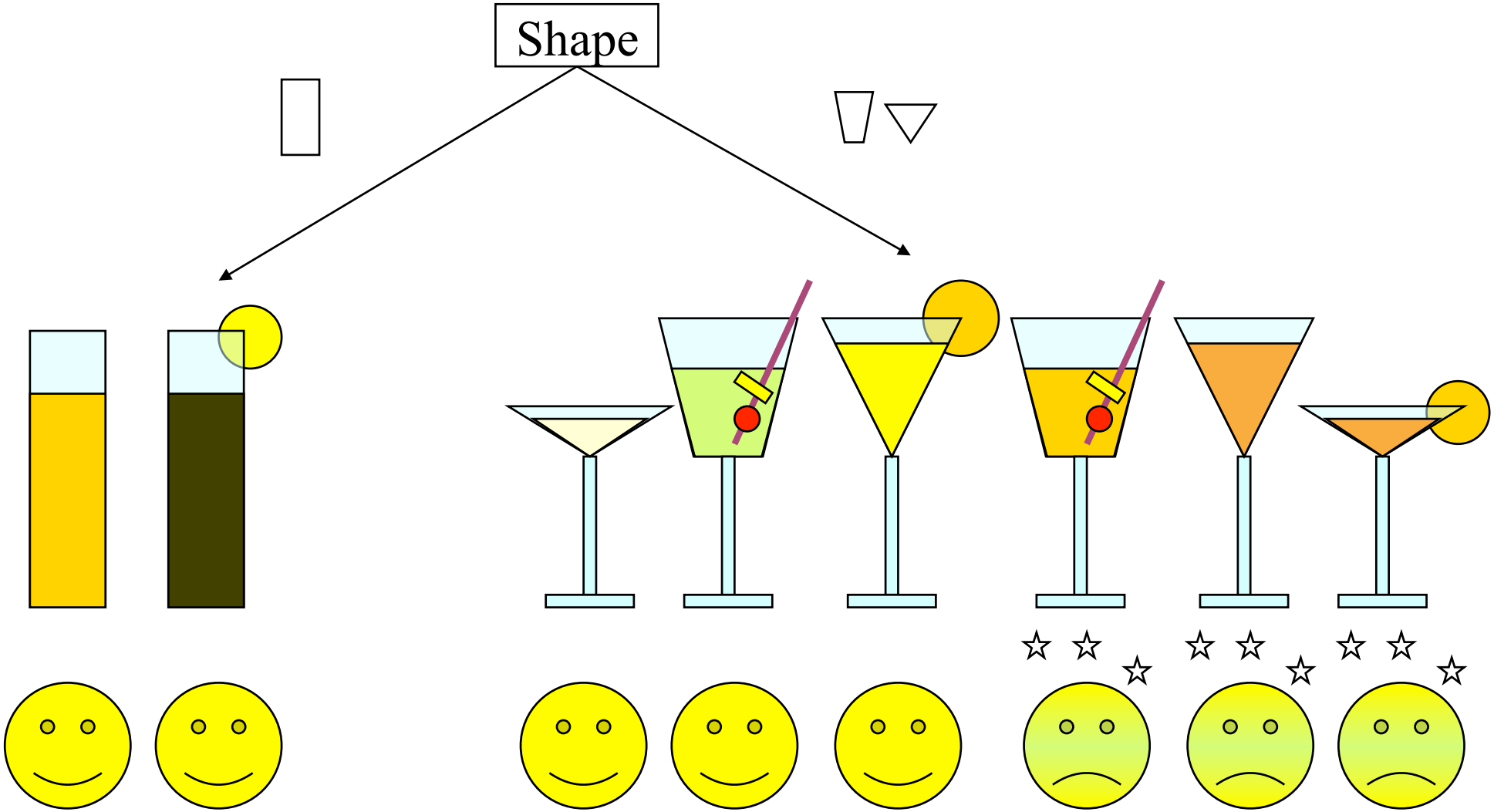
Data set in tabular form



Shape	Color	Content	Sick?
Cilinder	Orange	25cl	No
Cilinder	Black	25cl	No
Coupe	White	10cl	No
Trapezoid	Green	15cl	No
Coupe	Yellow	15cl	No
Trapezoid	Orange	15cl	Yes
Coupe	Orange	15cl	Yes
Coupe	Orange	10cl	Yes

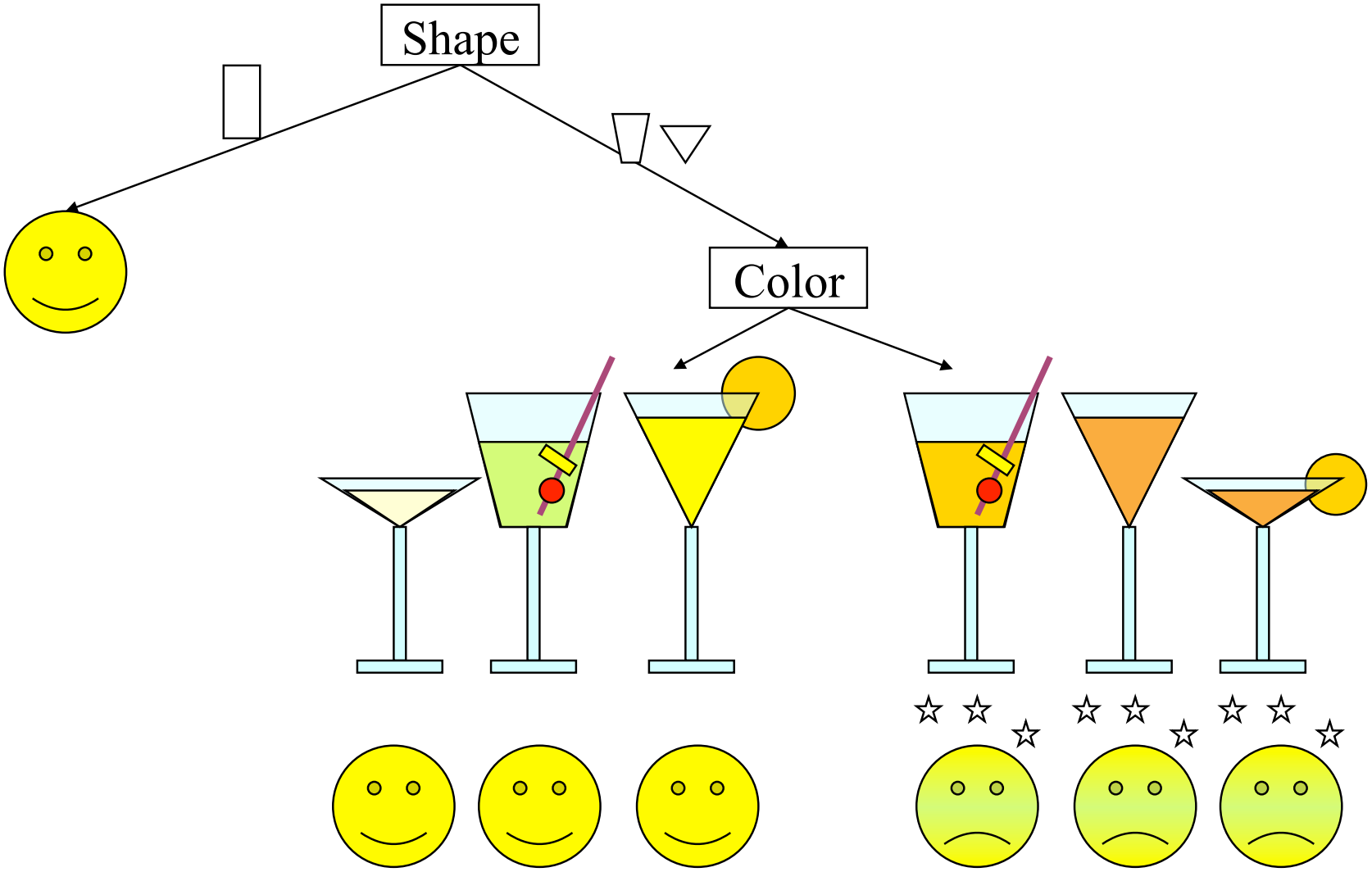
KU LEUVEN

Observation 1: shape is important



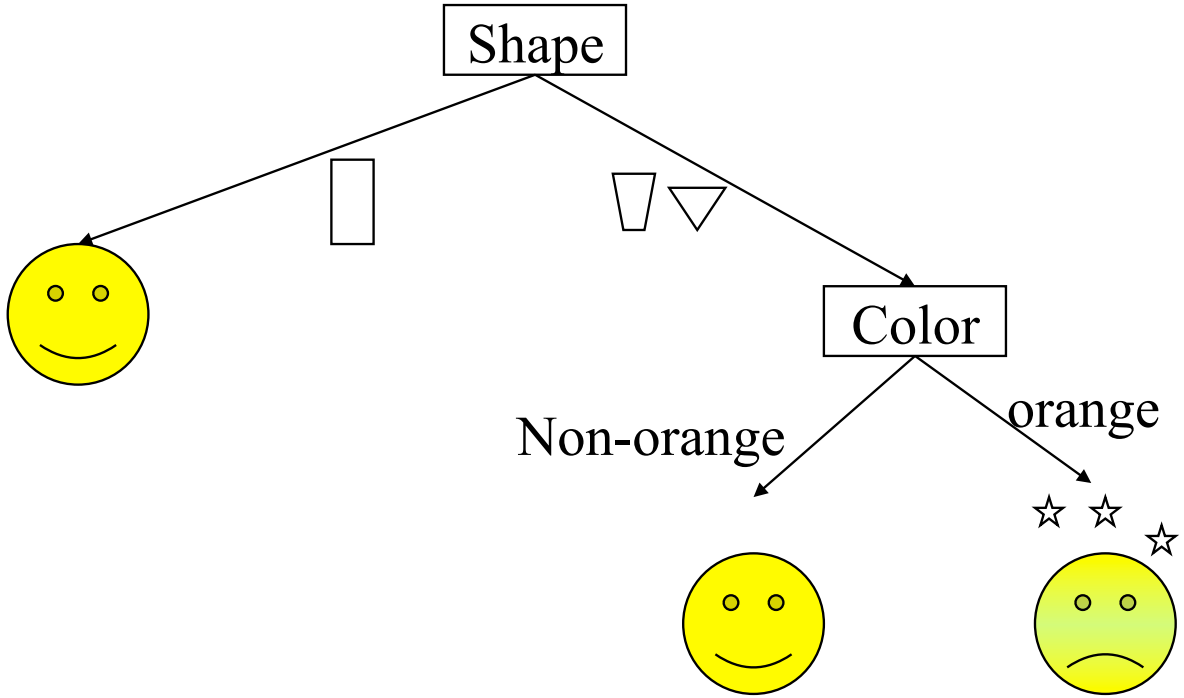
KU LEUVEN

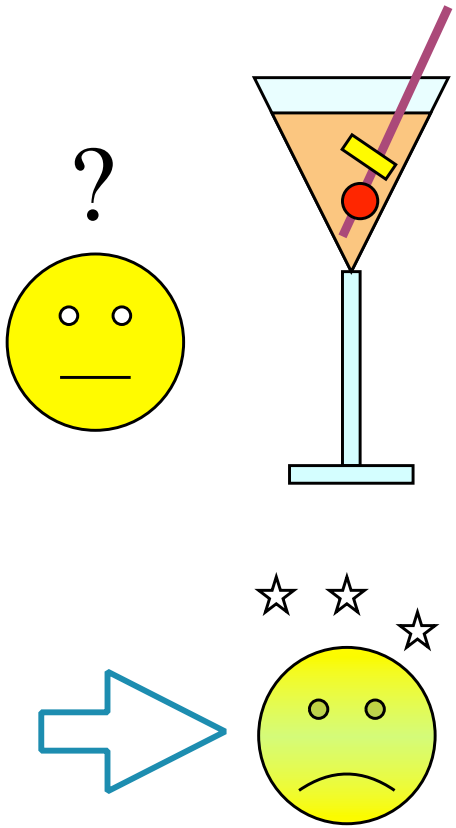
Observation 2: Sometimes, color is important too



KU LEUVEN

The decision tree



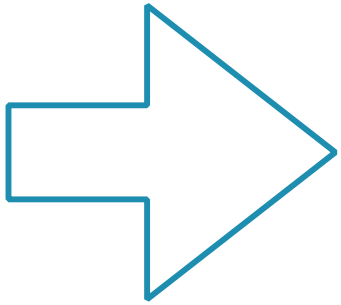


KU LEUVEN

Rule representation of tree

- Trees can be written as if-then-rules
- Rules can be simplified

if shape = cilindric then 😀
else
    if color ≠ orange then 😀
    else 🤢



if shape ≠ cilindric
  and color = orange
then 🤢
else 😀

KU LEUVEN

Two main questions…

- The basic principle has just been illustrated
- Two important details need to be filled in:
  - How to choose the “best” test
  - When to stop splitting nodes

KU LEUVEN

How to choose the tests?

- Think of the game “20 questions”, where you need to guess which person another player is thinking of, by asking questions that can only be answered by “yes” or “no”.
- What’s a good criterion for choosing the questions?

KU LEUVEN

Choosing a test: classification

- We focus on classification trees (Y is nominal)
- A good test is a test that carries much information about the class
- What is information?
- Information theory defines “entropy” or “missing information”
  - how many bits needed, on average, to convey a piece of information, if an optimal encoding is used

KU LEUVEN

Information

- Assume I need to tell you for 100 objects what their class is, out of four possibilities (A,B,C,D)
- Could encode A as 00, B as 01, C as 10, D as 11: 2 bits per objects needed
- Can we do better?
- If all classes are equally likely: no
- If some classes are frequent and others are rare: yes! Use shorter encoding for more frequent class.
  - E.g. A: 0, B: 10, C: 110, D: 111
  - 25% each: on average (1+2+3+3)/4 = 2.25 bits (worse than 2)
  - 50% A, 30%B, 10%C, 10%D: 0.5 + 0.6 + 0.3 + 0.3 = 1.7 bits

Information

- More clever encodings can be devised
- Given a set of values $c_1, c_2, \ldots, c_k$ with respective probabilities $p_1, p_2, \ldots, p_k$, an encoding exists that uses, on average, e bits for representing a randomly drawn value, where

$$e = -\sum_{i=1}^{k} p_i \log_2(p_i)$$

(with $\log_2$ the binary logarithm: $a = \log_2(b) \Leftrightarrow b = 2^a$)

- This number e reflects the minimal number of bits that you will need, on average, to encode one value. It is the inherent information content, or **entropy**

KU LEUVEN

Class entropy

- The **class entropy** of a set S of objects $(\boldsymbol{x}, y)$, where y can be any of k classes c_i, is defined as

$$CE(S) = -\sum_{i=1}^{k} p_i \log_2(p_i) \qquad \text{with } p_i = \frac{|\{(\boldsymbol{x}, y) \in S \,|\, y = c_i\}|}{|S|}$$

(proportion of elements in S with class c_i)

- It measures how much uncertainty there is about the class of a particular instance

KU LEUVEN

Entropy: example

- S contains 6 elements with classes A, A, B, B, B, C:
  - CE(S) = - 1/3 $\log_2$ 1/3 - 1/2 $\log_2$ 1/2 - 1/6 $\log_2$ 1/6 = 1.46
- S contains 16 elements with 8xA, 8xB:
  - CE(S) = - 1/2 $\log_2$ 1/2 - 1/2 $\log_2$ 1/2 = 1
- S contains 16 elements, 15xA, 1xB:
  - CE(S) = -15/16 $\log_2$ 15/16 - 1/16 $\log_2$ 1/16 = 0.34

- High entropy = “many possibilities, all equally likely”
- Low entropy = “few possibilities”, or possibly: “many possibilities, but most are highly unlikely”
- **Entropy measures uncertainty**

Information gain

- Entropy reflects missing information
- The **information gain** of a question = the expected reduction of entropy by obtaining the answer to the question
- In the case of classification trees: expected reduction of class entropy:

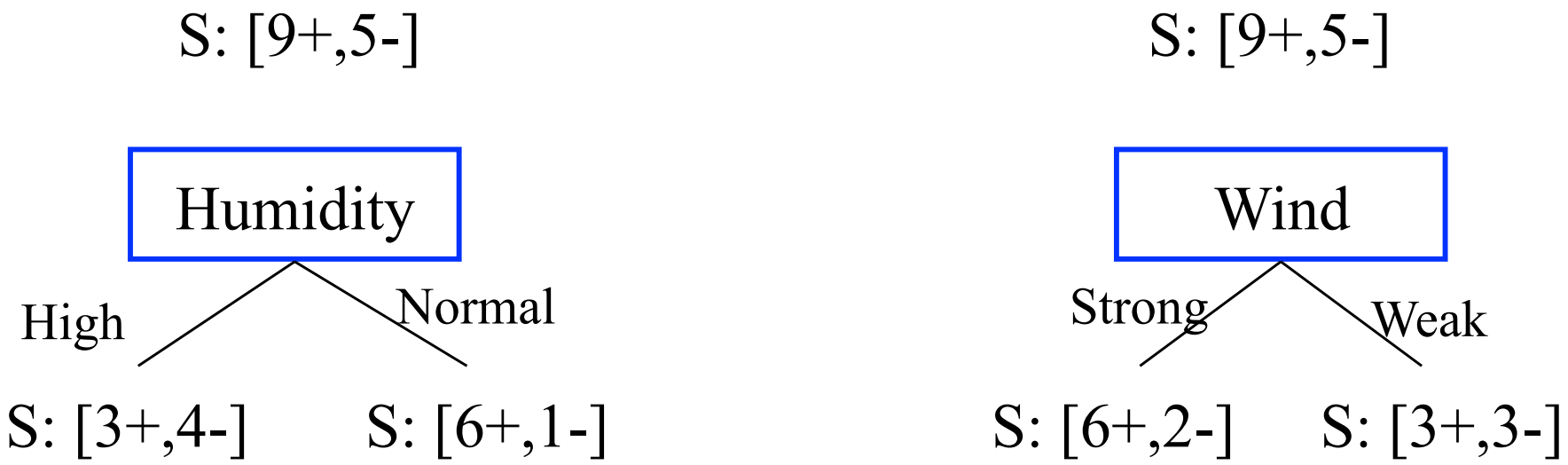
$$IG(S,t) = CE(S) - \mathbf{E}[CE(S_i)] = CE(S) - \sum_{i=1}^{o} \frac{|S_i|}{|S|} CE(S_i)$$

with t a test, o the number of possible outcomes of t, and S_i the subset of S for which the i'th outcome was obtained

KU LEUVEN

Example

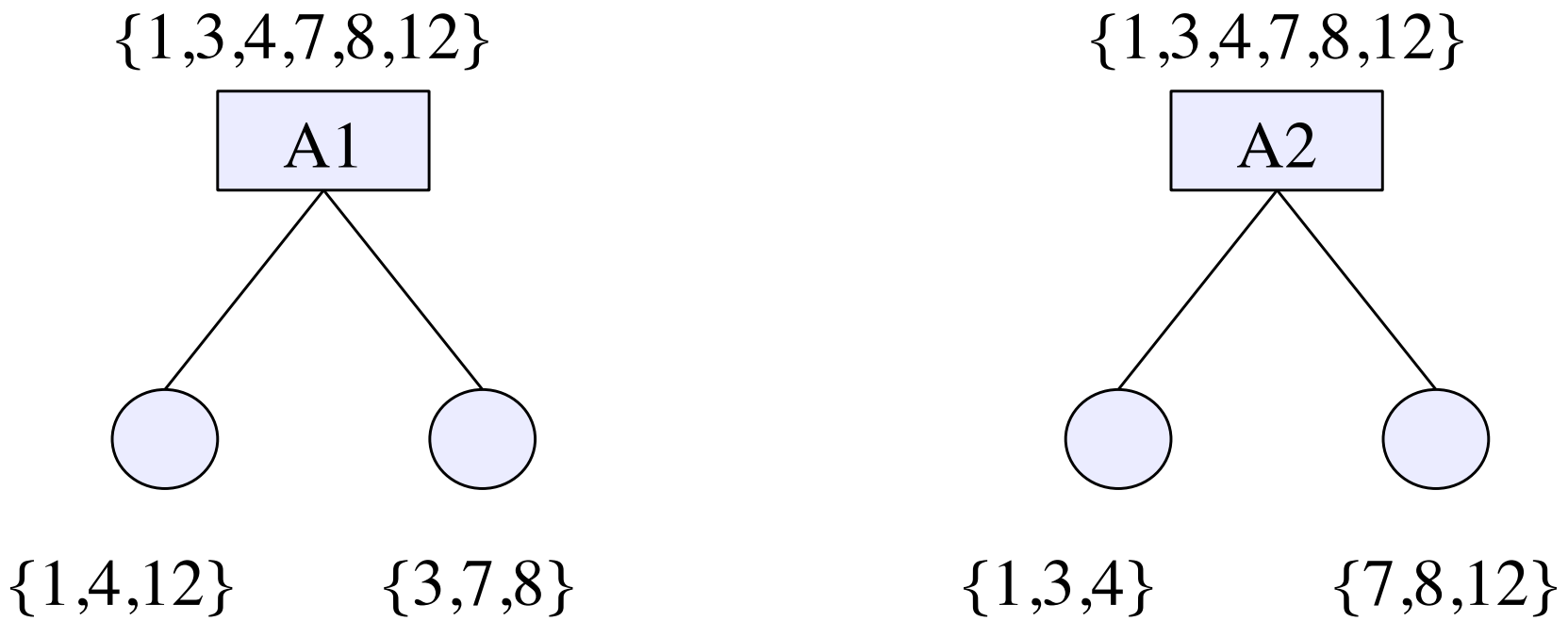
Tennis: assume set S has 9 “yes” (+) and 5 “no” (-)examples



KU LEUVEN

Choosing a test: regression

- Now assume Y is numerical
- How would you now choose the test?



Variance reduction

- Try to create subsets such that examples in one subset have similar Y values
- Instead of information gain: variance reduction

- The variance of Y in a set S of instances $(\boldsymbol{x}, y)$ is

$$Var(S) = \frac{\sum_{(\boldsymbol{x},y)\in S} (y - \bar{y})^2}{|S| - 1} \qquad \text{with } \bar{y} = \frac{\sum_{(\boldsymbol{x},y)\in S} y}{|S|}$$

- Variance reduction is defined similarly to information gain:

$$VR(S,t) = Var(S) - \sum_{i=1}^{o} \frac{|S_i|}{|S|} Var(S_i)$$

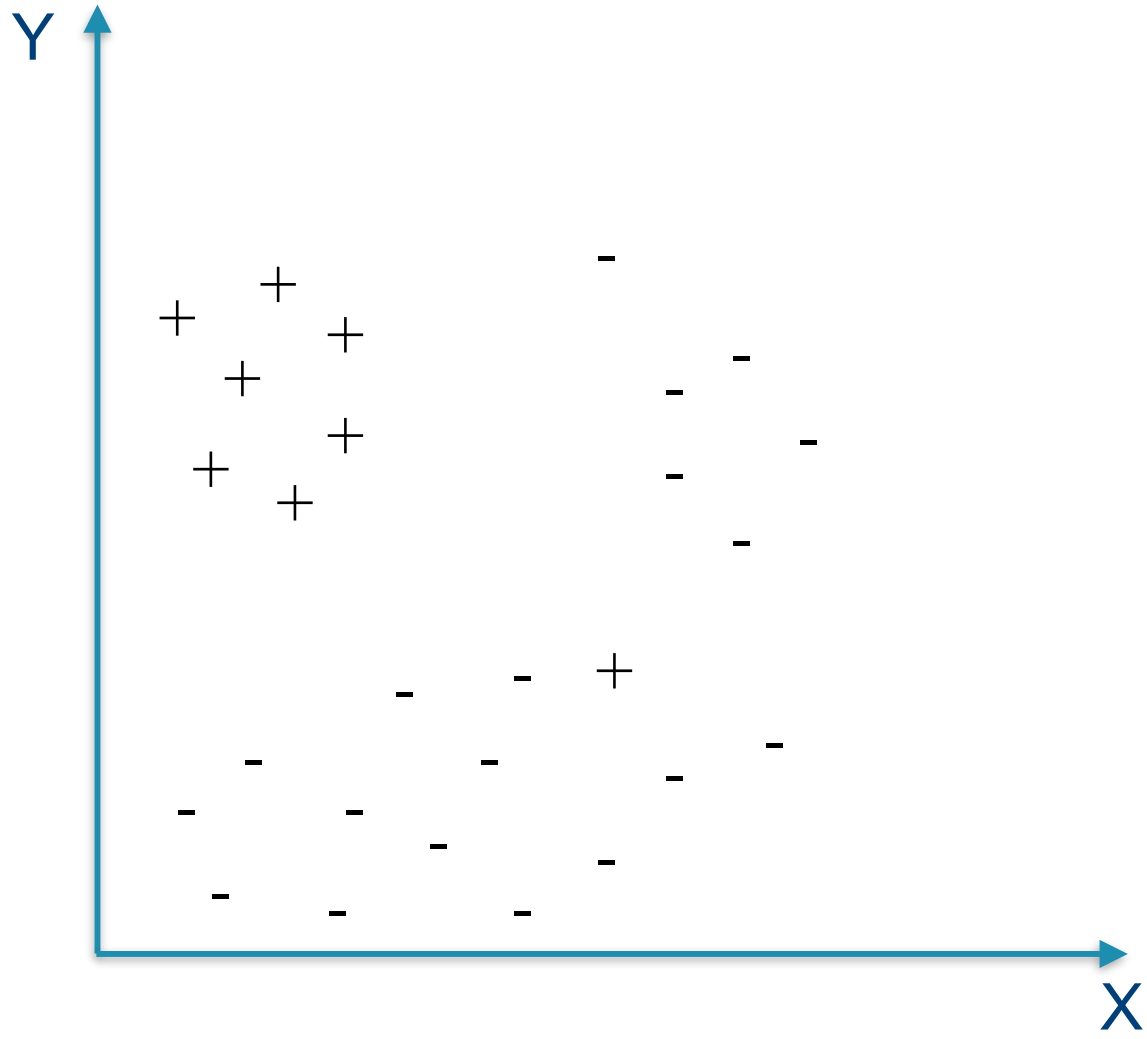
KU LEUVEN

When to stop splitting nodes?

- In principle, keep splitting until all instances in a subset have the same Y value
- Useful for Task 1 (no generalization needed)
- Less useful for Task 2: risk of overfitting

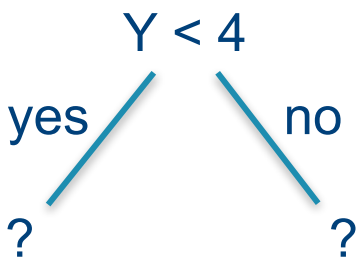
KU LEUVEN

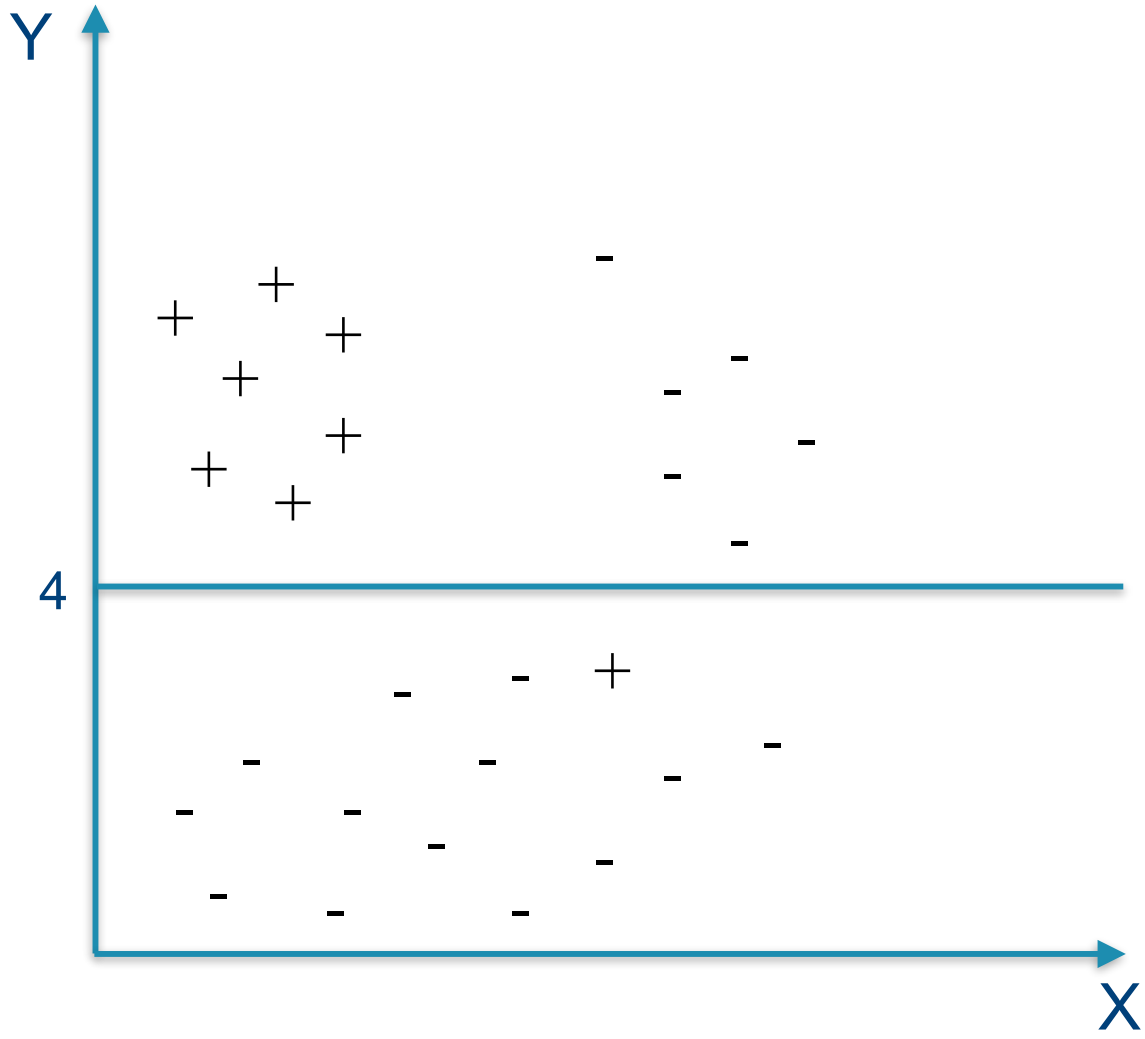
Tree with tests of the form X<… or Y<…



KU LEUVEN

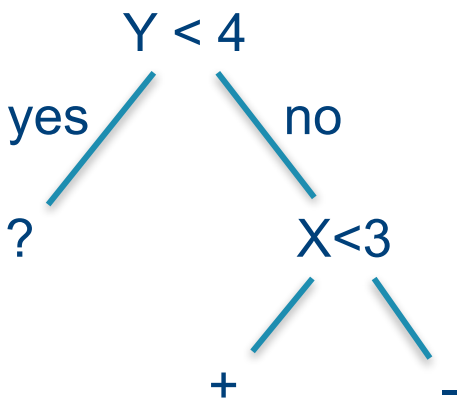
Tree with tests of the form X<… or Y<…

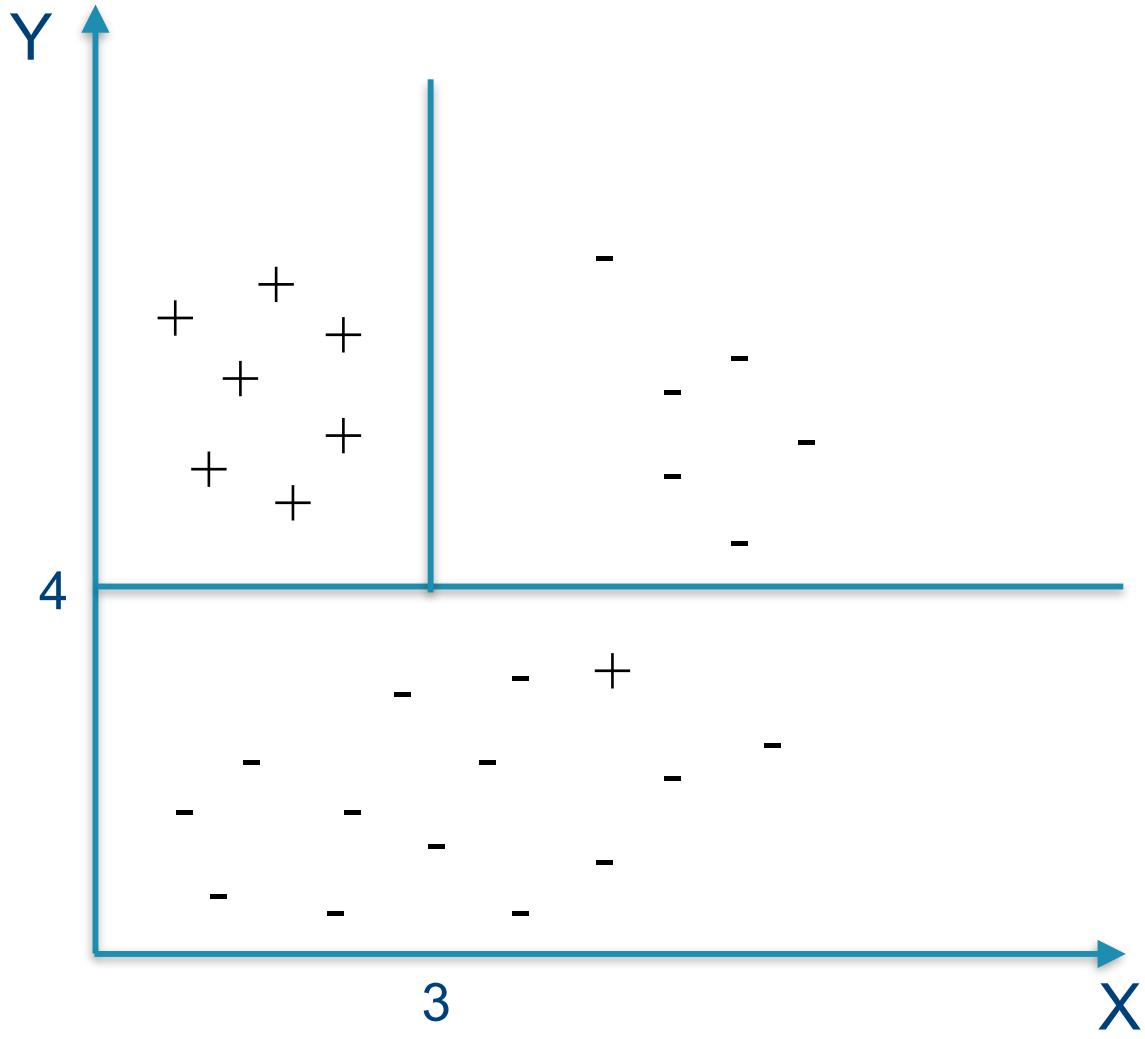




KU LEUVEN

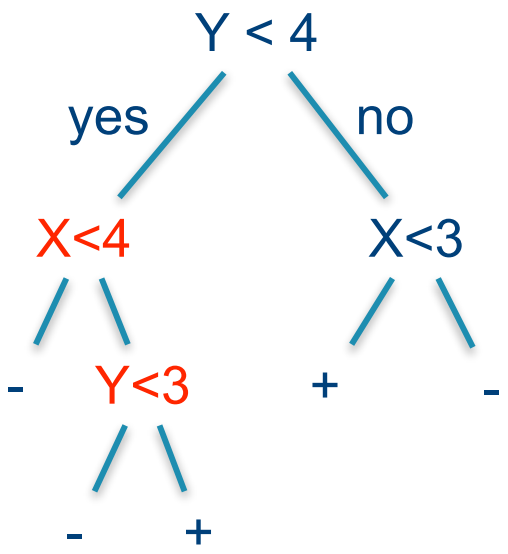
Tree with tests of the form X<… or Y<…

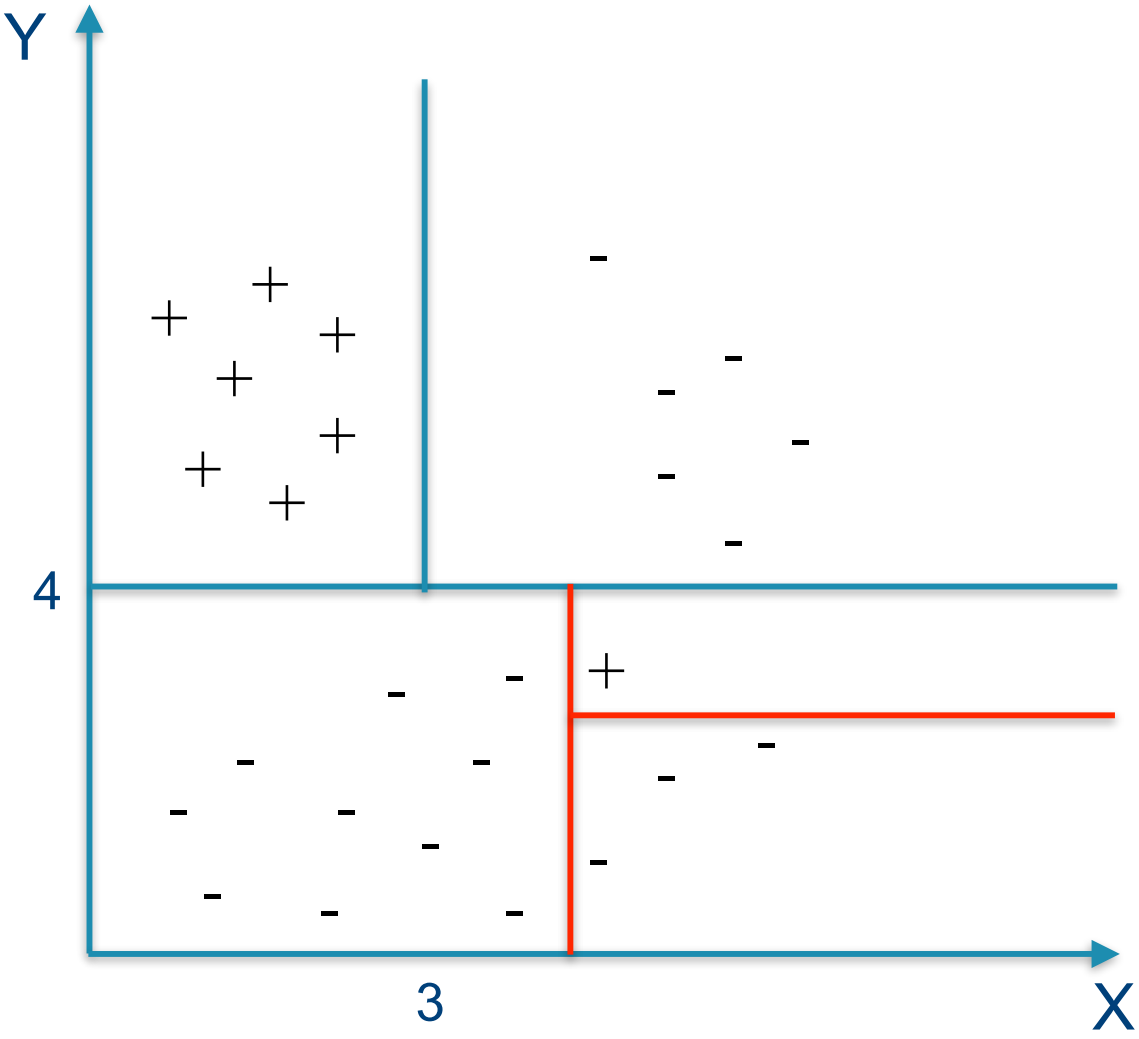




KU LEUVEN

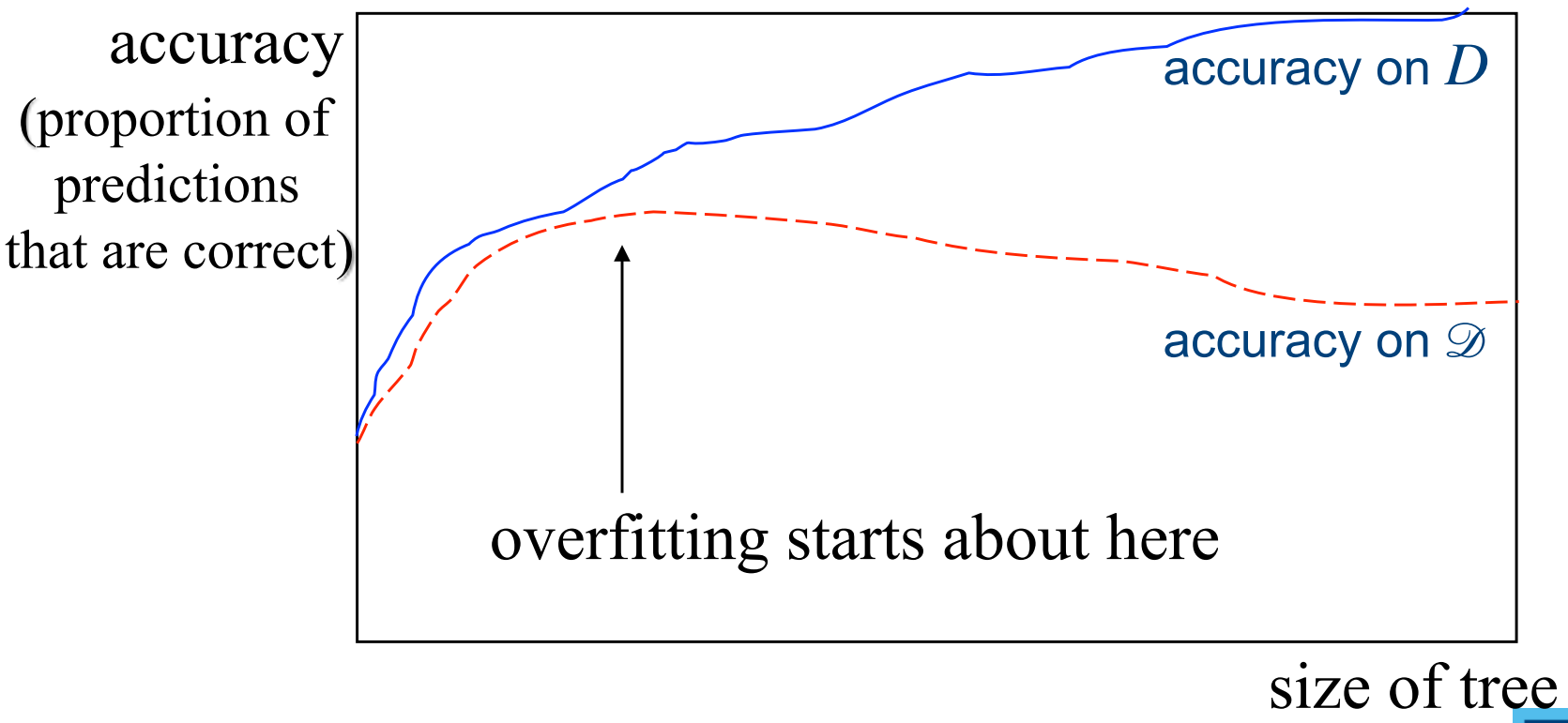
Tree with tests of the form X<… or Y<…





Overfitting: effect on generalization

- Overfitting improves the consistency of the tree with the given data set D, but may decrease accuracy for instances outside D



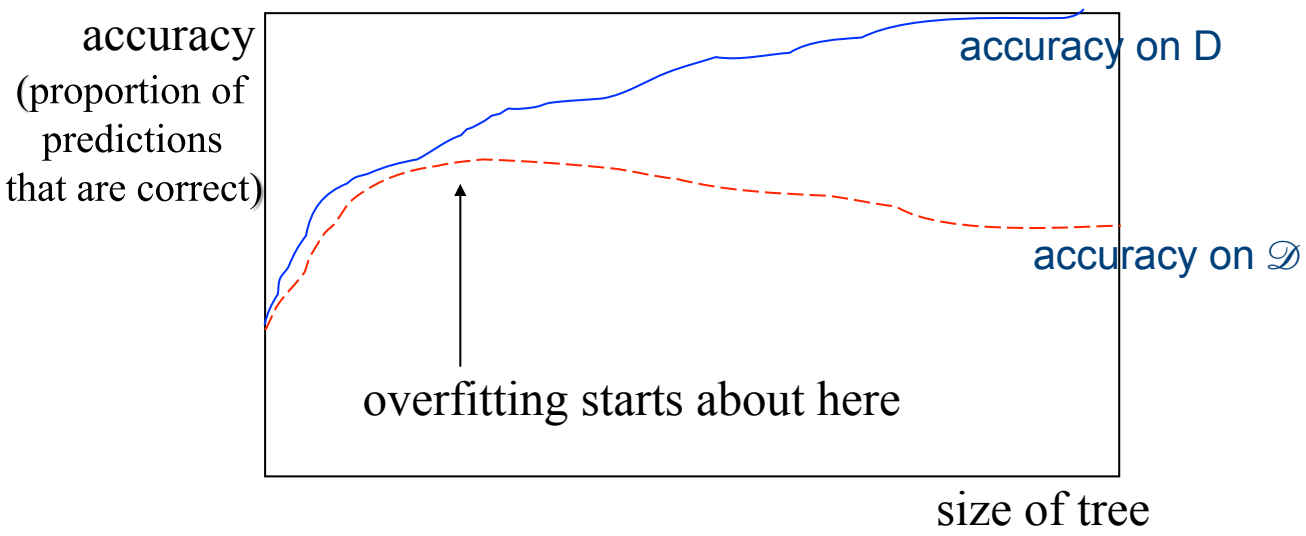
How to avoid overfitting?

Two options:

1. “cautious splitting”: do not split a node unless you are certain that the split is meaningful (will not lead to overfitting)
2. “post-pruning”: don’t bother about overfitting while splitting nodes, but once the (large) tree has been built, prune away branches that turned out not to contribute much

1. Cautious splitting

- How do we know when not to split a node any further?
- Multiple criteria possible
- A simple approach: try to guess when accuracy on unseen data is going down (the “turning point” in the red curve)



- But how can we guess this turning point, if the red curve is based on “unseen data”?
- Solution: “validation set”
  - set aside a subset of the data
  - this set is not used for growing the tree, only for estimating accuracy on “unseen” data

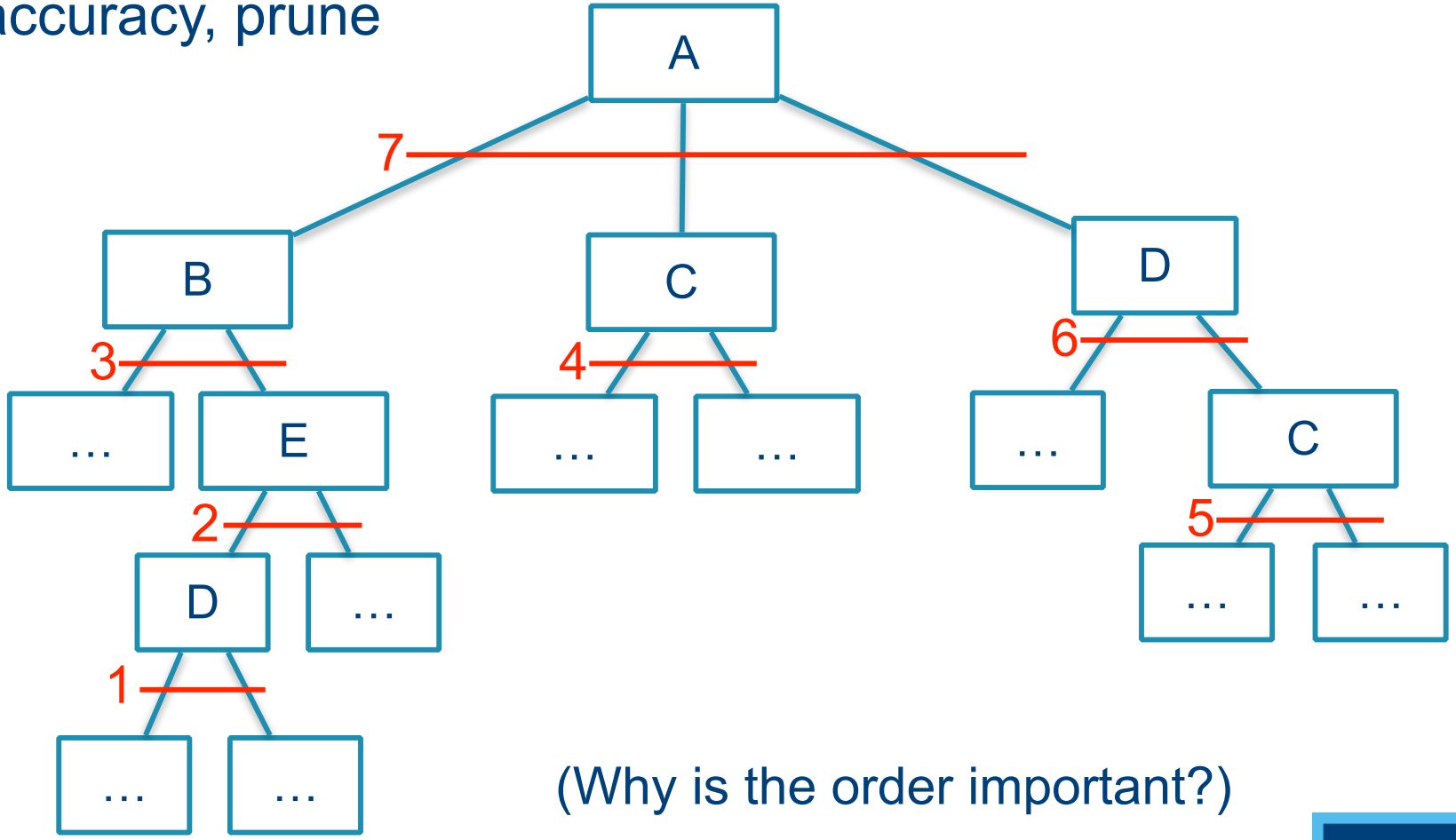
KU LEUVEN

2. Post-pruning

- What if the red curve has a dip, then rises again?
- Risk of ending the growth phase too early
- Possible solution: compute the whole red curve in advance, then find its highest point
- “Post-pruning” principle:
  - grow the tree to its full size, then cut away branches that did not contribute to getting better predictions
  - Specifically: check for each node in the tree, starting at the bottom: “what would be the accuracy (on the validation set) if I cut the tree here?”
  - If that accuracy is not lower, cut the tree

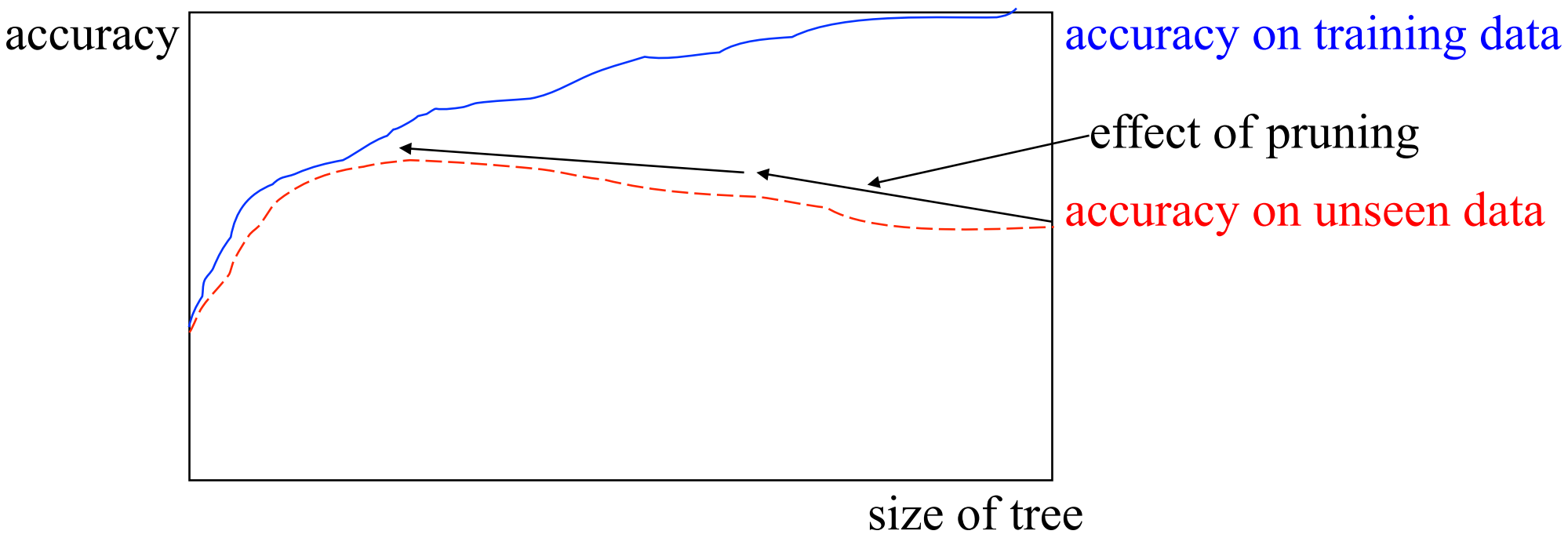
Illustration

- Try pruning subtrees *in the indicated order*; whenever no loss of accuracy, prune



(Why is the order important?)

KU LEUVEN


accuracy
accuracy on training data
effect of pruning
accuracy on unseen data
size of tree

Post-pruning vs. cautious splitting

- Post-pruning requires more effort (build a large tree, then throw away most of it), but tends to give more accurate trees (for the reasons mentioned)
- Not pruning at all is usually a bad idea (overfitting)

KU LEUVEN



A generic algorithm

The common structure of DT learners
The differences between them

KU LEUVEN

A generic TDIDT algorithm

- TDIDT = “top-down induction of decision trees”, also referred to as “recursive partitioning”

- Most decision tree learners follow the same basic approach, but differ in the details
- We will have a look at the commonalities and differences

KU LEUVEN

A generic TDIDT algorithm

```
function TDIDT(E: set of examples) returns tree;
    T' = grow_tree(E );
    T = prune_tree(T');
    return T;

function grow_tree(E: set of examples) returns tree;
    T = generate_tests(E);
    t = best_test(T, E);                          (call t's outcomes v1…vk)
    P = {E1, E2, …, Ek} with Ei={x∈E | t(x)=vi}   (P = partition induced on E by t)
    if stop_criterion(E, P)
    then return leaf(info(E))
    else
        for all Ei in P: Ti := grow_tree(Ei);
        return node(t, {(v1,T1), (v2, T2), … (vk, Tk)} );
```


KU LEUVEN

A generic TDIDT algorithm

The blue functions are where implementations differ

- prune_tree : how to prune the tree afterwards (see before)
- generate_tests : which tests to consider
- best_test : which test to select (uses heuristics)
- stop_criterion : when to stop
- info : what information to store in the leaf

KU LEUVEN

generate_tests

Up till now, we assumed

- one subtree per value for nominal attributes
- two subtrees for numerical attributes ($X<c$)

Alternative for nominal attributes with many values:

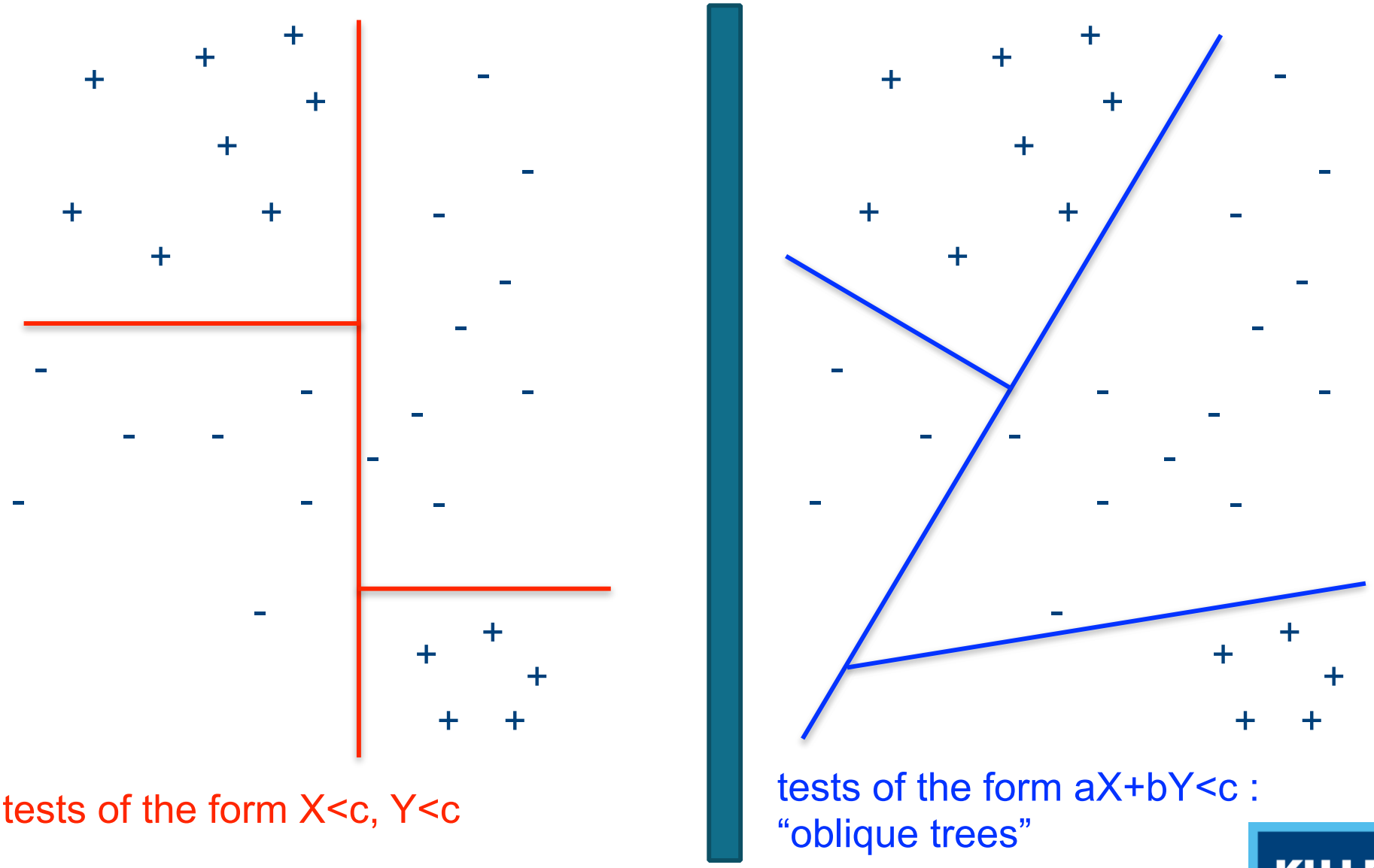
- one subtree per *set* of values

For numerical attributes:

- how to determine c?
- alternatives: tests of the form $X \in [c_1, c_2]$, or $aX_1+bX_2<c$, or …

KU LEUVEN

Illustration: effect of type of tests



tests of the form X<c, Y<c

tests of the form aX+bY<c :
“oblique trees”

Type of tests affects expressiveness

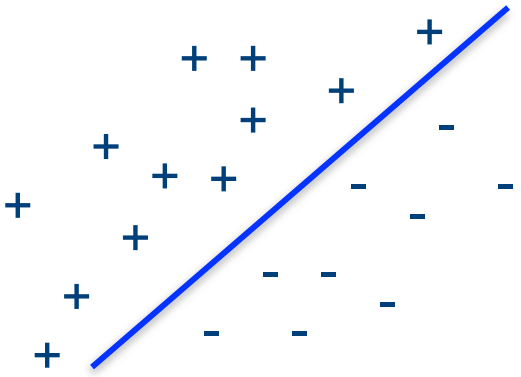
Oblique trees are more expressive…

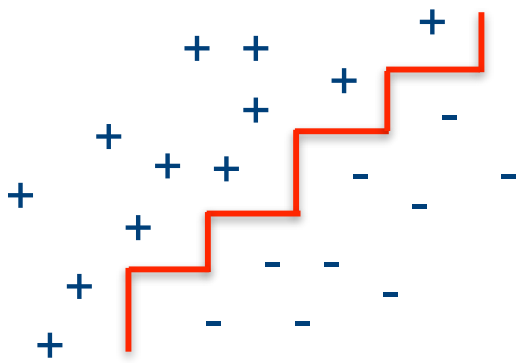
- easier to represent “diagonal” class boundaries

… but also much harder to learn

- the number of tests to consider is much higher
- optimal test is harder to determine (requires optimization of many parameters)

In practice, non-oblique trees are much more common





KU LEUVEN

best_test

For classification trees:

- information gain = reduction of entropy: $CE(S) = -\sum_{i=1}^{k} p_i \log p_i$
- alternatives based on information theory have been proposed, e.g., C4.5 / J48 use *gain ratio* (see next slide)
- CART reduces "Gini impurity" instead of entropy: $Gini(S) = 1 - \sum_{i=1}^{k} p_i^2$

For regression trees:

- reduction of variance, reduction of standard deviation, …

Gain ratio heuristic

- Tests with many possible outcomes have an (unfair?) advantage over tests with few outcomes
  - cf. “20 questions”: only yes/no answers allowed. Questions with many possible answers, such as “what’s their profession?” or even “what’s their first name?” make the game too easy
- To make the comparison between binary and non-binary tests fairer, “normalize” the gain by dividing it by some kind of maximum potential gain: the “split information” (SI)
- For a test t that splits S into n subsets S_i :

$$SI(S,t) = -\sum_{i=1}^{n} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$ (Note: this is the “t-entropy” in S; cf. class entropy)

- The Gain Ratio is: $GR(S,t) = IG(S,t)/SI(S,t)$

KU LEUVEN

Numerical inputs

- Typically a binary split of the form x < c is made
- So for one attribute, we have many possible tests, depending on how we fill in c
- How to determine c?
- Typically, all values are tried, and the c that yields the best heuristic value (IG, GR, Gni, …) is selected
- Complexity? Not as bad as you might think - see later

KU LEUVEN

info

For classification trees:

- usually the most frequent class in this leaf
- can also store the class distribution, e.g. “30%A, 60%B, 10%C”
- some variants store all training examples relevant for that leaf

For regression trees:

- usually the mean of all target values in that leaf
- possible alternatives: median, …

stop_criterion

As explained before, post-pruning is usually preferred over stopping early

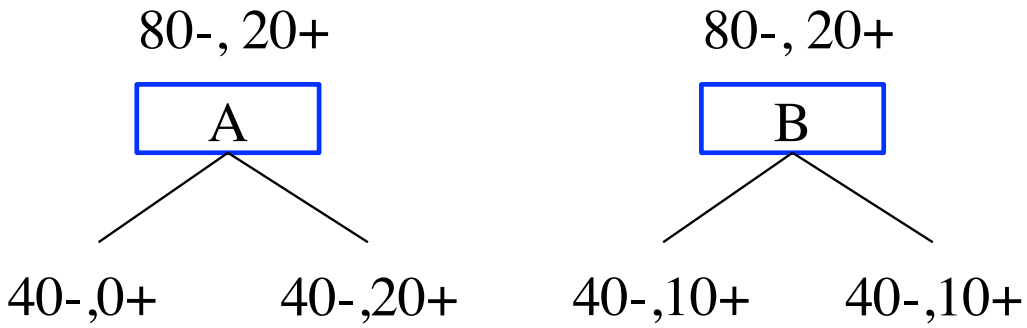
Still, there may be “obvious” cases where you can stop early:

- Classification: all instances have the same class
- Too few examples to continue splitting
  - From a generalization viewpoint, leaves of size one do not make sense
  - Therefore, only allow tests that yield subtrees with at least two examples each

KU LEUVEN

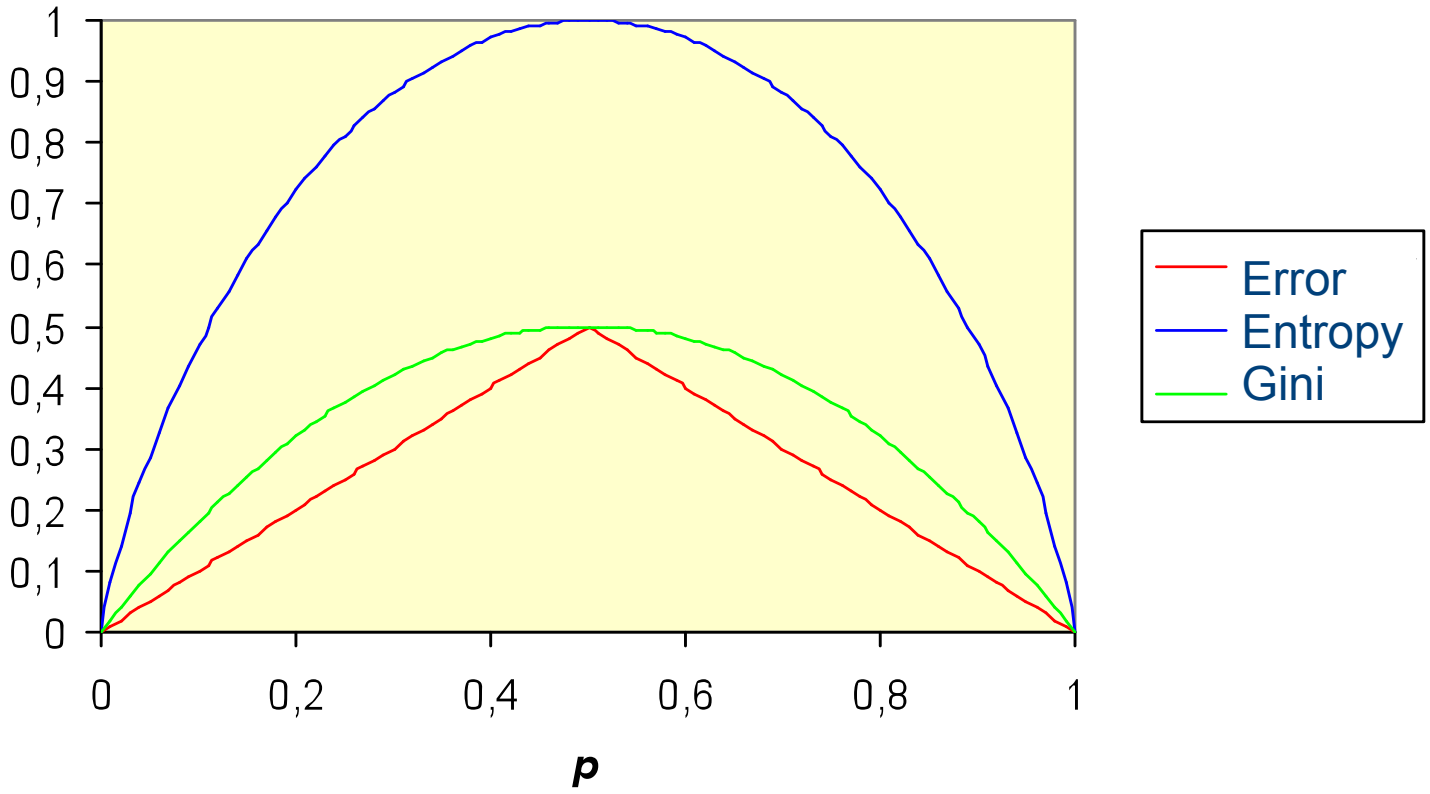
A note on heuristics for classification

- C4.5, J48 use entropy as impurity measure
- CART uses Gini as impurity measure
- Question: why not use a more obvious impurity measure, based on p_i instead of $p_i \, blog(p_i)$ or p_i^2 ? For instance: predictive accuracy of expanded tree
- This actually has been proposed by some. But it does not work well.



- Which test gives the most accurate tree?
- Which is best according to Gini or IG?

Impurity measures

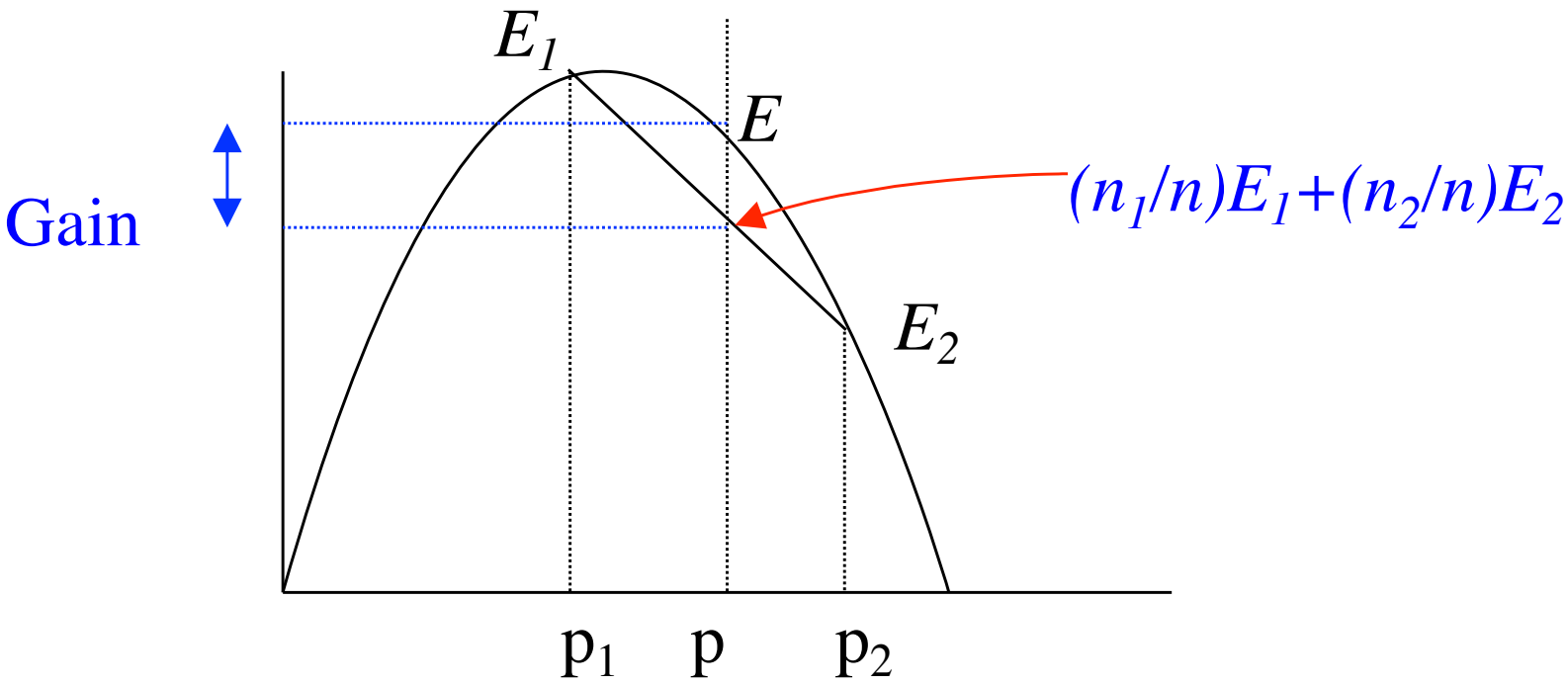


Impurity as a function of p, when there are only two classes

Observation: *good impurity measures are strictly concave*

KU LEUVEN

Good impurity measures are strictly concave (Breiman et al., 1984)



Assume node with size n, entropy E and proportion of positives p is split into 2 nodes with n_1, E_1, p_1 and n_2, E_2 p_2.

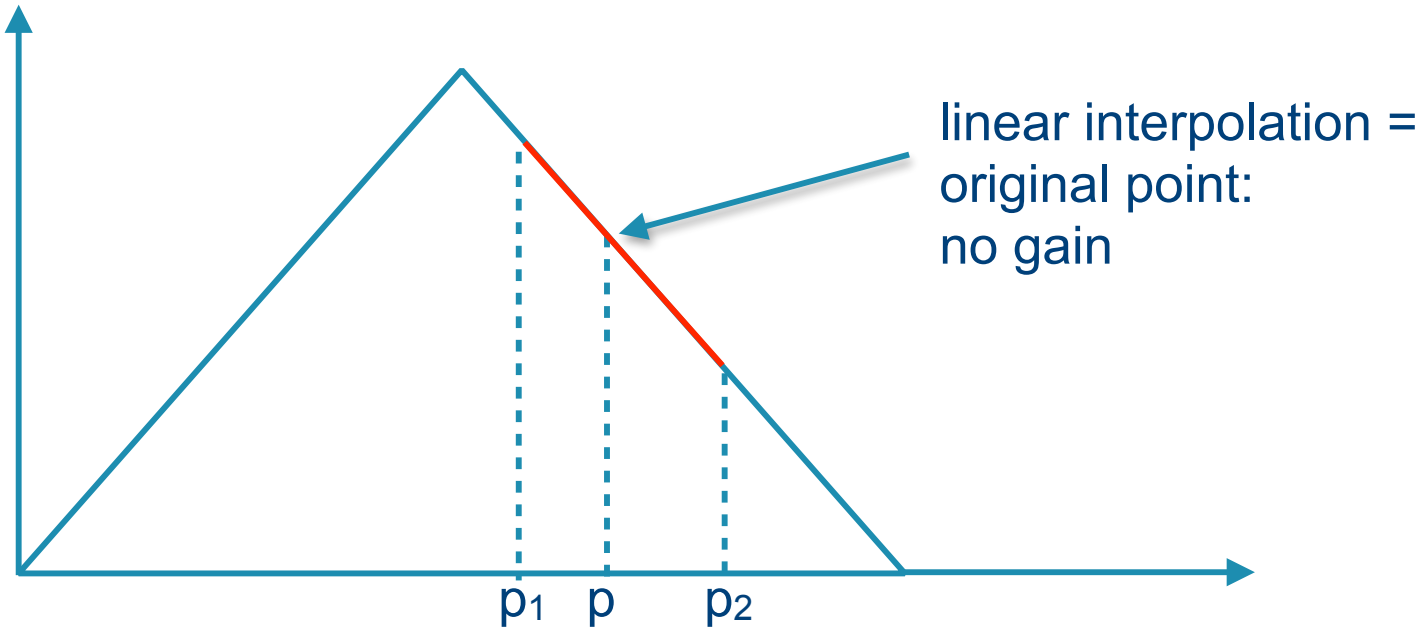
We have $p = (n_1/n)p_1 + (n_2/n)\, p_2$,

so the average entropy $E' = (n_1/n)E_1+(n_2/n)E_2$ is found by linear interpolation between (p_1,E_1) and (p_2,E_2) at p.

Information gain = difference in height between (p, E) and (p,E').

KU LEUVEN

Good impurity measures are strictly concave

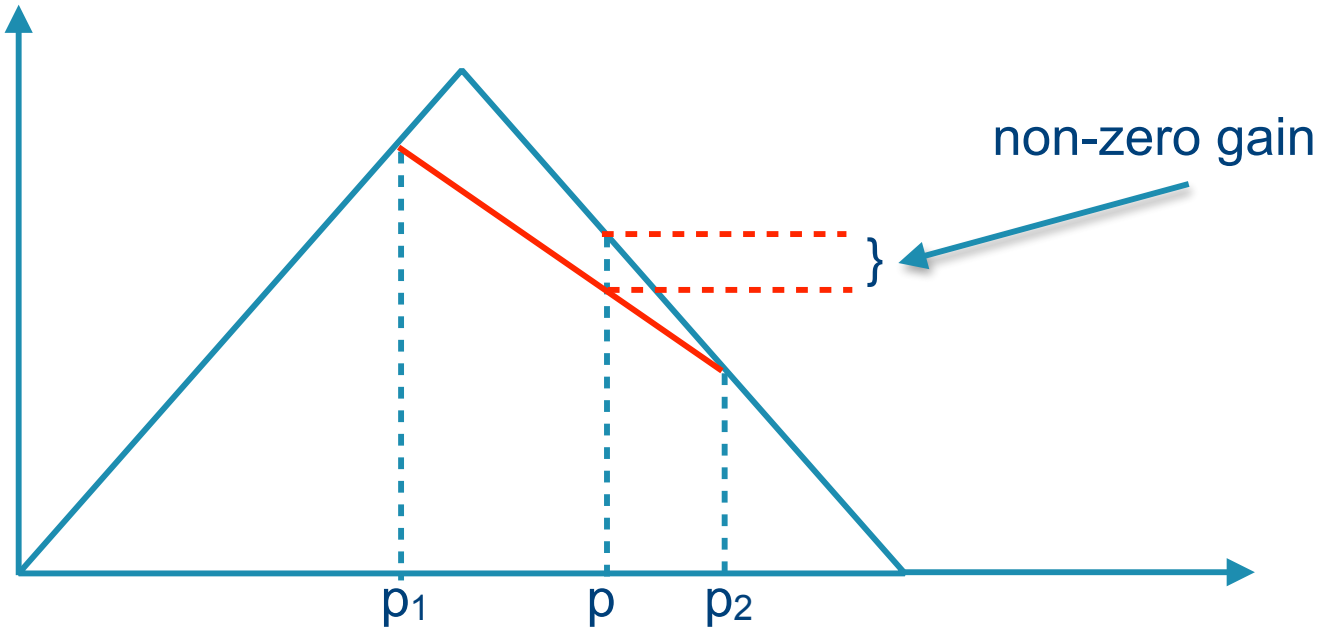


Accuracy will only improve if at least one child node has a majority class different from that of its parent

This is often too restrictive: many good tests yield 0 accuracy gain

KU LEUVEN

Good impurity measures are strictly concave



Accuracy will only improve if at least one child node has a majority class different from that of its parent

This is often too restrictive: many good tests yield 0 accuracy gain

KU LEUVEN



How much work is it?

Computational complexity of learning and using a decision tree

KU LEUVEN

Computational complexity of tree learning

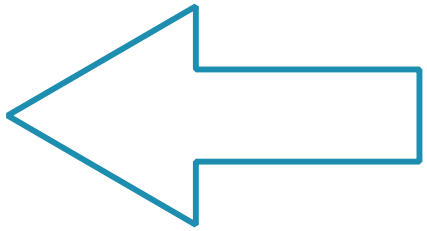
- Given a data set D consisting of N instances (**x**,y), where **x** has m components (attributes)
- How do N and m influence the computational effort required to learn a tree?

KU LEUVEN

Splitting one node

For each node, we need to “find the best test” :

- for each possible test, evaluate its quality
  - partition the dataset according to this test
  - compute quality based on this partition
- for the test with highest quality:
  - partition data according to that test



KU LEUVEN

Computing the quality of one test

- Consider information gain = reduction of entropy
- We need to compute class entropy for the node and for its subsets
- This means we need to compute the proportions for each class in each subset

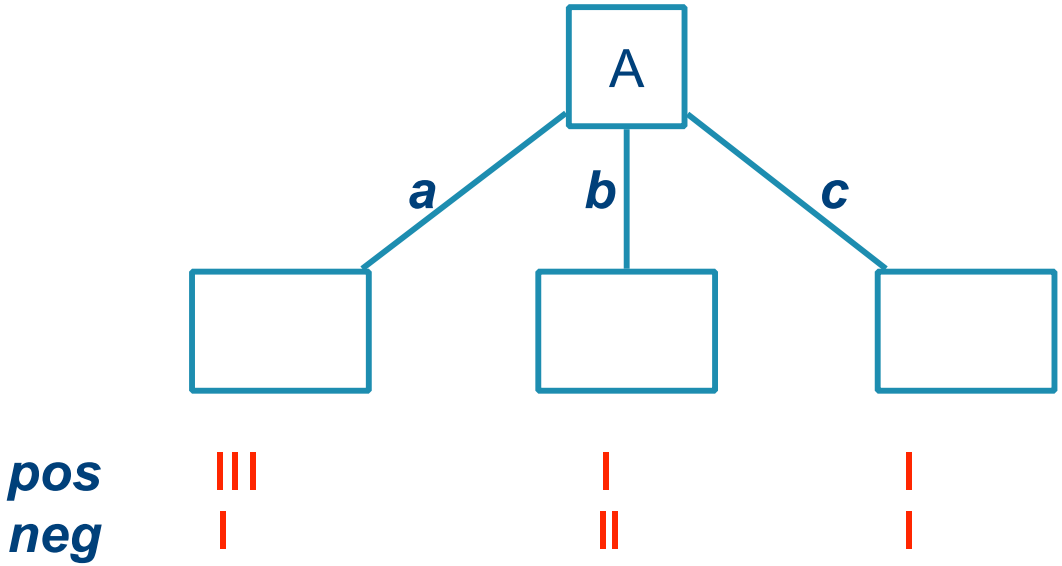
$$IG(S,t) = CE(S) - \sum_{i=1}^{o} \frac{|S_i|}{|S|} CE(S_i)$$

$$CE(S) = -\sum_{i=1}^{k} p_i \,\mathrm{blog}(p_i) \qquad \text{with } p_i = \frac{|\{(\mathbf{x}, y) \in S | y = C_i\}|}{|S|}$$

KU LEUVEN

Computing the quality of one test

A	B	C	Class
a			pos
b			neg
a			pos
c			pos
b			neg
c			neg
a			pos
b			pos
a			neg



counts	a	b	c
pos	3	1	1
neg	1	2	1
all	4	3	2

for c:
$p_{pos}=0.5$
$p_{neg}=0.5$

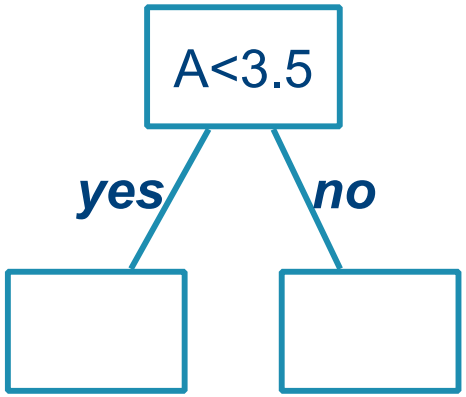
$|S_i|$

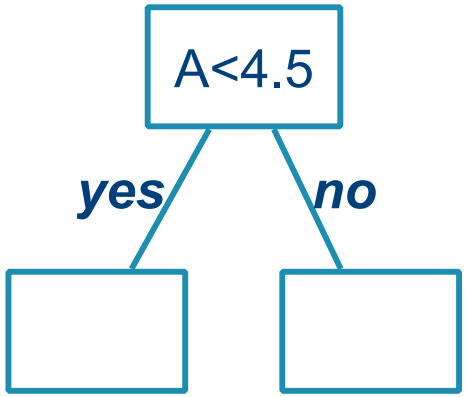
KU LEUVEN

Computing the quality of one test: continuous attributes

A	B	C	Class
3			pos
4			neg
3			pos
5			pos
7			neg
8			neg
6			pos
3			pos
2			neg

How to choose the threshold?
Evaluate all possibilities? A lot of work!





counts	yes	no
pos	3	2
neg	1	3
all	4	5

counts	yes	no
pos	3	2
neg	2	2
all	5	4

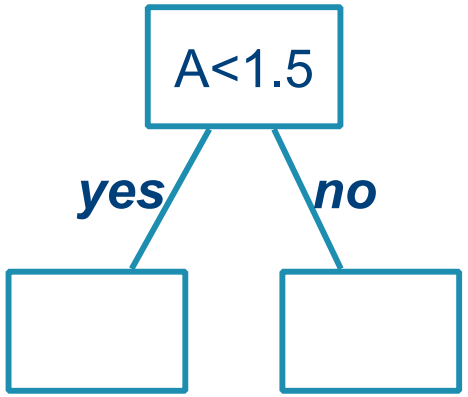
… and so on, for 2.5, 5.5, 6.5, 7.5

KU LEUVEN

Efficiently computing test quality for continuous attributes

- First sort all tuples according to A, small to large
- We can then gradually move the threshold, and simply update the tables
- Thus, testing all thresholds is only slightly more work than testing one!

A	B	C	Class
2			neg
3			pos
3			pos
3			pos
4			neg
5			pos
6			pos
7			neg
8			neg



counts	yes	no
pos	0	5
neg	0	4
all	0	9

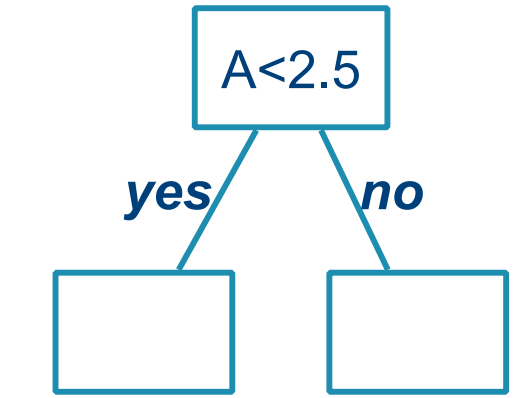
$|S_{yes}|=0$, $|S_{no}|=9$
no: $p_{pos}=5/9$, $p_{neg}=4/9$

KU LEUVEN

Efficiently computing test quality for continuous attributes

- First sort all tuples according to A, small to large
- We can then gradually move the threshold, and simply update the tables
- Thus, testing all thresholds is only slightly more work than testing one!

A	B	C	Class
2			neg
3			pos
3			pos
3			pos
4			neg
5			pos
6			pos
7			neg
8			neg



counts	yes	no
pos	0	5
neg	1	3
all	1	8

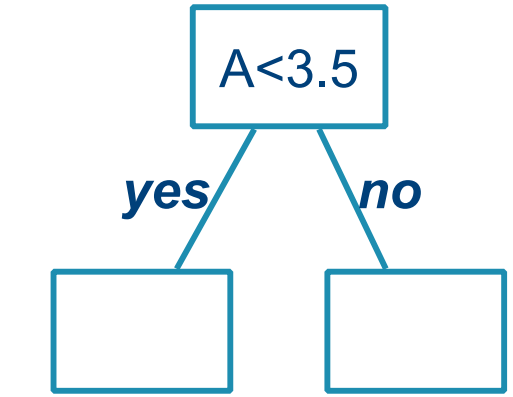
$|S_{yes}|=1$, $|S_{no}|=8$
no: $p_{pos}=5/8$, $p_{neg}=3/8$
yes: …

KU LEUVEN

Efficiently computing test quality for continuous attributes

- First sort all tuples according to A, small to large
- We can then gradually move the threshold, and simply update the tables
- Thus, testing **all** thresholds is **only slightly more work** than testing one!

A	B	C	Class
2			neg
3			pos
3			pos
3			pos
4			neg
5			pos
6			pos
7			neg
8			neg



counts	yes	no
pos	3	2
neg	1	3
all	4	5

$|S_{yes}|=1$, $|S_{no}|=8$
no: $p_{pos}=2/5$, $p_{neg}=3/5$
yes: …

KU LEUVEN

Computing the quality of one test

- For nominal attributes: 1 scan, gradually building the table; compute IG once, at the end
- For continuous attributes: 1 scan, gradually updating the table; compute IG for each intermediate table. (This assumes we've sorted the dataset according to this attribute - but that can be done once, in the beginning, we don't need to redo that each time.)
- Hence, continuous is a bit more work (calculating IG multiple times), but only by a constant factor
- *Computing the best test for a node is linear in the number of instances in that node*

KU LEUVEN

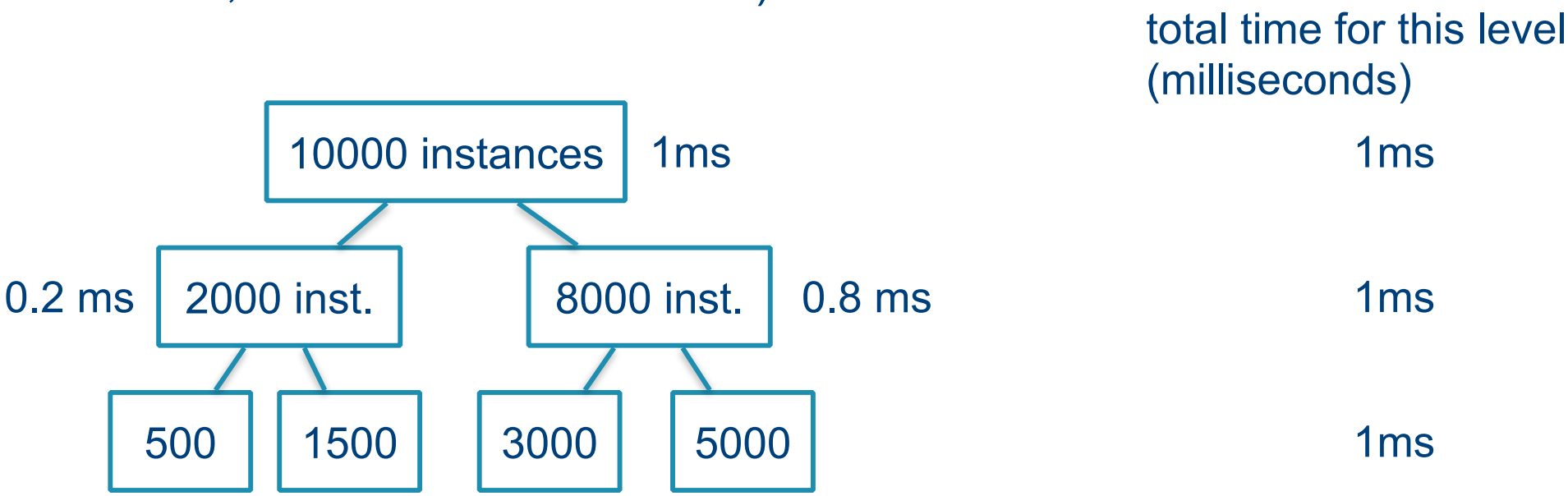
Splitting one node

- Let n be the number of instances *in that node* (not the total number of instances N)
- In practice, usually:
  - #tests is proportional to #attributes m
  - “compute quality of test” is O(n)
- Compute quality of all tests is then O(nm)

KU LEUVEN

Splitting multiple nodes

- After splitting a node into multiple children, we have more nodes, but each of them has fewer data elements
- The total at one level of the tree is always N
- So, the overall amount of work stays the same for each level in the tree (or diminishes, when leaves are created)



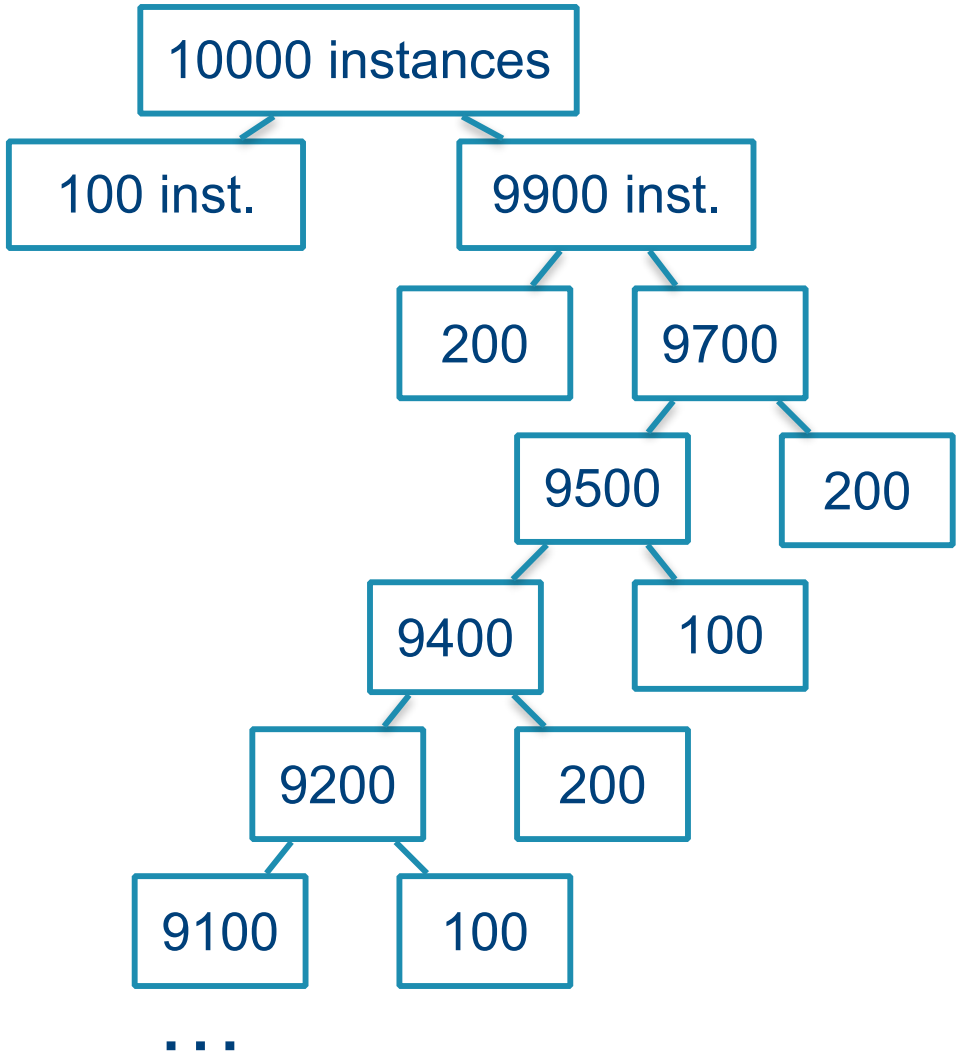
Overall complexity

- If splits are balanced, height of tree is O(log(N))
- Then overall complexity of tree growing is O(m N log(N)), with
  - N total number of instances in the dataset
  - m total number of attributes

- Under these assumptions, tree learning has very good complexity in #instances and #attributes!
- However, can be $O(m N^2)$ if splits are unbalanced

KU LEUVEN

Overall complexity: unbalanced trees



For unbalanced trees, height can be O(N)…

This can happen in practice!

Good heuristics (e.g., IG) tend to favor balanced splits.

KU LEUVEN

Making predictions

Using a tree to make predictions is usually very fast: h tests, with h the height of the tree

Tree learners try to find trees with small height

- remember: heuristics try to reduce entropy or variance
- the faster you can reduce these, the faster you find “pure” sets, with (near) zero variance/entropy, which become leaves

So, decision trees are very efficient at prediction time

KU LEUVEN

Complexity: conclusions

- Under mild assumptions, decision tree learning is very efficient
- That's an excellent property when learning from big data !

- *Prediction* using decision trees is *extremely* fast (<1µs)

KU LEUVEN

Handling missing values



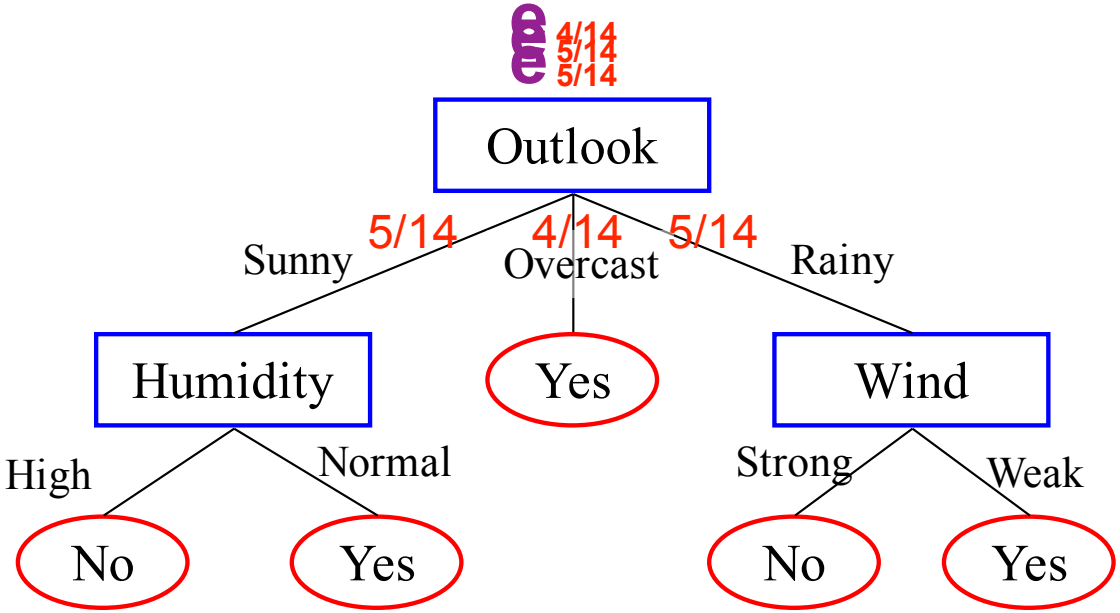
KU LEUVEN

What about missing values?

- When *computing quality* of a given test: just ignore instances where this value is missing
- When *splitting* on an attribute, where should an example with a missing value go? Two options:
  - 1: guess its most likely value, then choose the corresponding branch (*imputation*)
  - 2: *partially* assign the example to multiple branches, proportional to probability of each branch (*distribution*)
- When making a *prediction*: weighted vote among all leaves where the example (partially) ends up

Distribution: illustration

	Outlook	Hum.	Wind	Play
1	Sunny	High	Weak	No
2	sunny	high	strong	No
3	overcast	High	weak	yes
4	rain	High	Weak	yes
5	rain	normal	weak	yes
6	rain	normal	strong	no
7	overcast	normal	strong	yes
8	sunny	high	weak	no
9	Sunny	normal	weak	yes
10	rain	normal	weak	yes
11	Sunny	normal	strong	yes
12	overcast	high	Strong	yes
13	overcast	normal	weak	yes
14	rain	high	strong	No



e: 10/14 no, 4/14 yes => guess no

How to classify **e : [15, ?, high, strong]** ?

KU LEUVEN



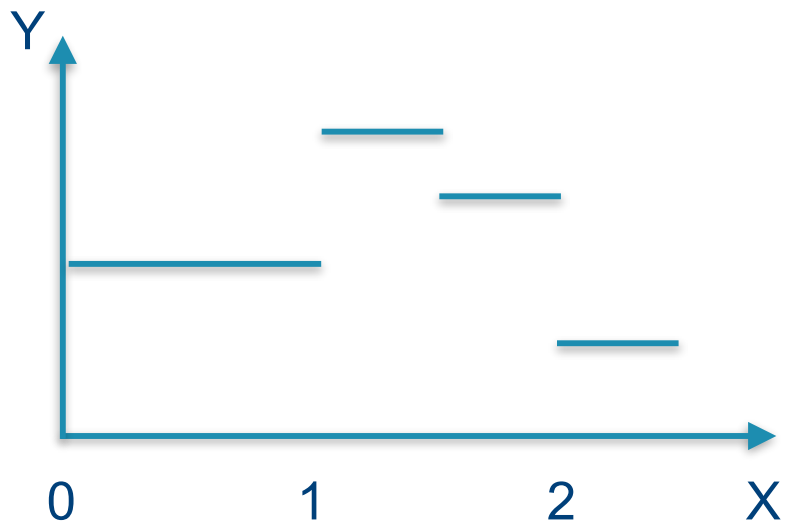
Model trees

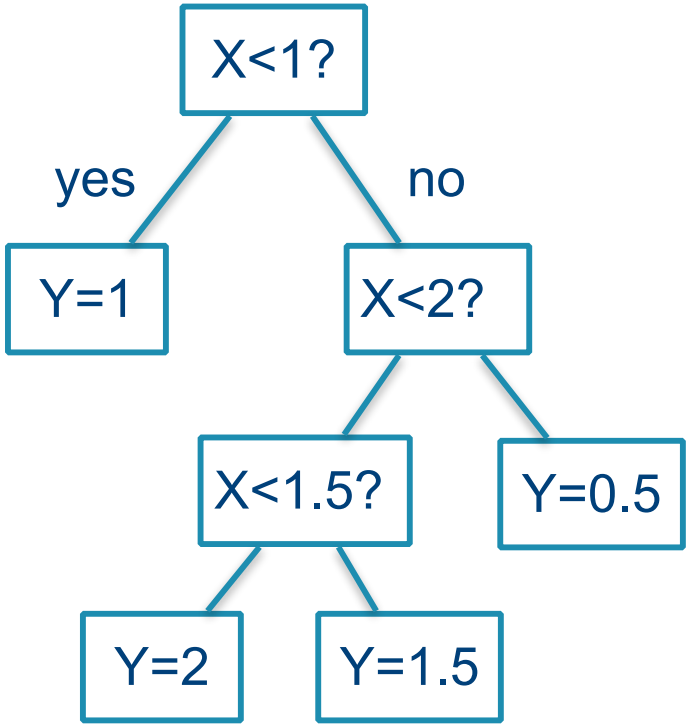
More complex versions of regression trees

KU LEUVEN

Representation power of regression trees

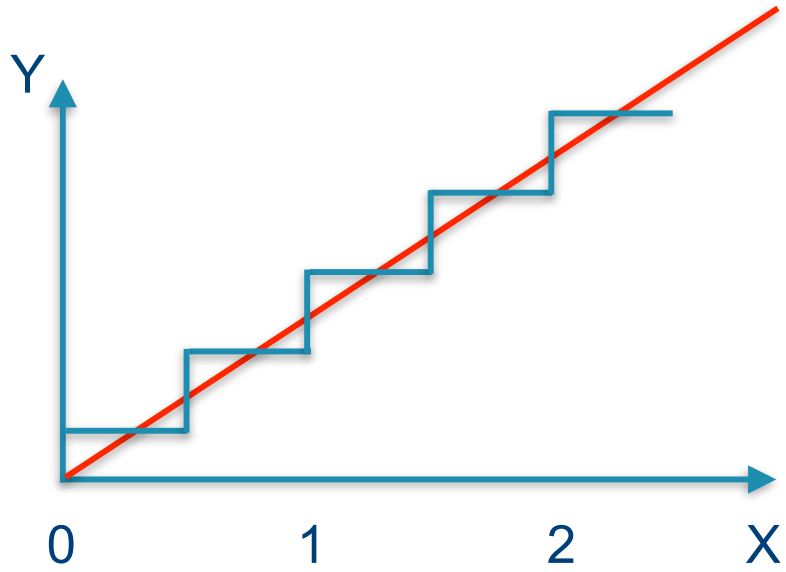
Regression trees represent *piecewise constant* functions

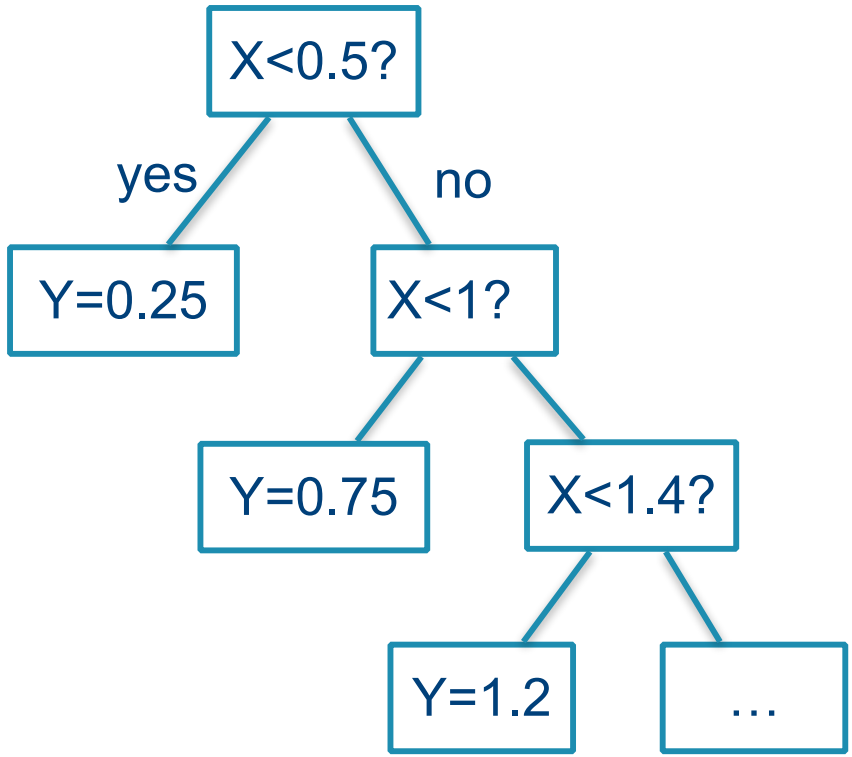




Representation power of regression trees

What if the target function is Y=X?

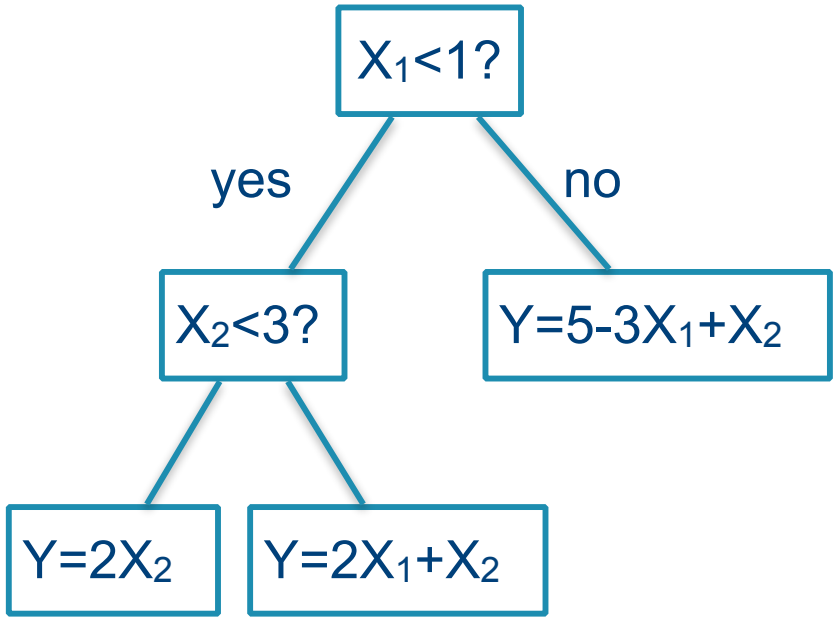




Model trees

Model trees are regression trees in which each leaf does not contain a constant, but a linear model

They represent *piecewise linear* functions



KU LEUVEN

A model tree learner: M5

Probably the best known model tree learner (Quinlan, 1992)

Weka implementation: M5' (Wang & Witten, 1997)

Standard regression tree learning, but when tree building ends, *builds a linear model in each leaf*.

Note : M5 still chooses test that *minimize variance*, but this is no longer optimal in this case (Why not? What would be better?)

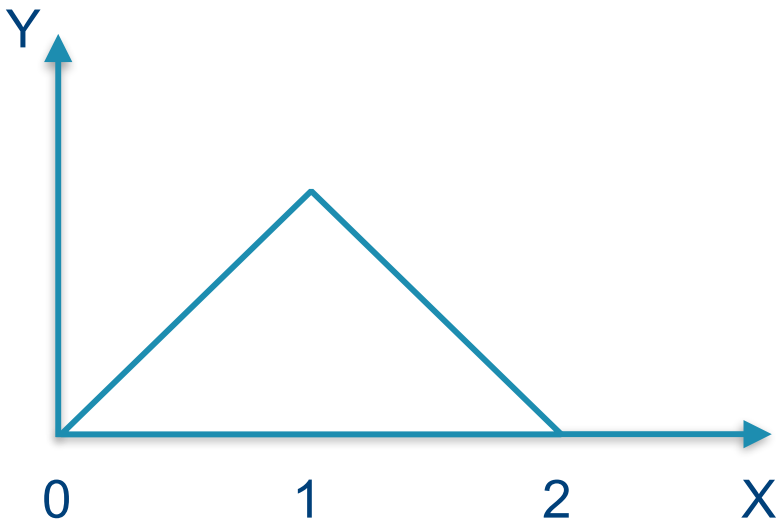
KU LEUVEN

RETIS

Observation:

- a subset is good if the data in that subset can be approximated well by a linear model
- the variance of Y is not necessarily a good indicator of that!

M5's heuristics make sense if we will have constant predictions in the leaves, not if we will have linear models making the predictions



What is a good split here,
- for regression trees ?
- for model trees ?

RETIS

A better criterion, when splitting into subsets, is to:

- perform linear regression in each subset
- compute the *residual* variance of this regression
- choose the test that minimizes the average residual variance

The RETIS system (Karalic, 1992) does exactly that.

The bad news is: deciding the optimal test is $O(m^3)$, with m the number of variables (due to multiple regression) (Torgo, 2002). This makes RETIS very slow for high m.

KU LEUVEN

Mauve

Model tree learner **Mauve** chooses tests based on univariate regression

- slightly slower than M5, but not as slow as RETIS
- smaller & more accurate trees than M5

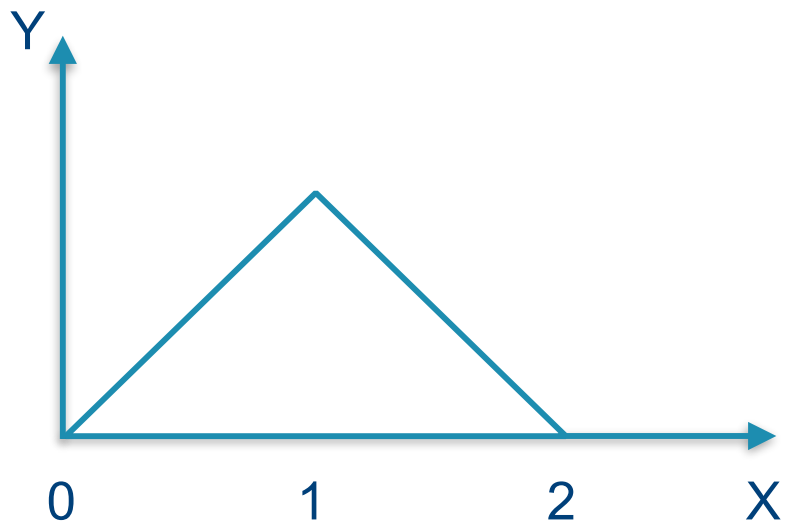
Basic idea: evaluate a test X<c based on reduction of residual variance after *univariate* linear regression (Y=aX+b)

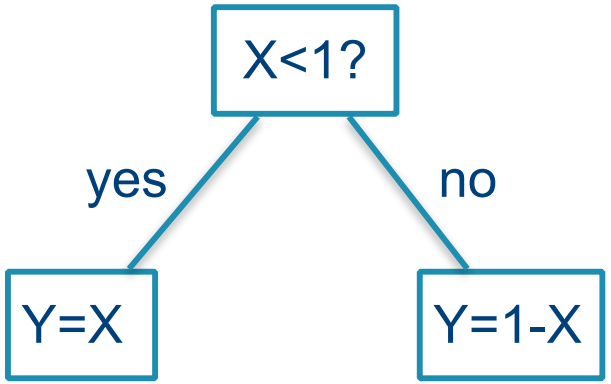
Vens et al. (2006): **A simple regression based heuristic for learning model trees**. Intelligent Data Analysis 10(3): 215-236.

KU LEUVEN

Mauve: motivating example

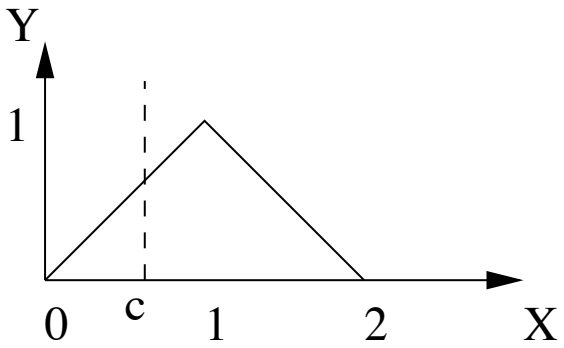
- Consider target function: f(x)=x if 0<x<1, 1-x for 1<x<2
- A tree exists that expresses it perfectly
- Does M5 find this tree?



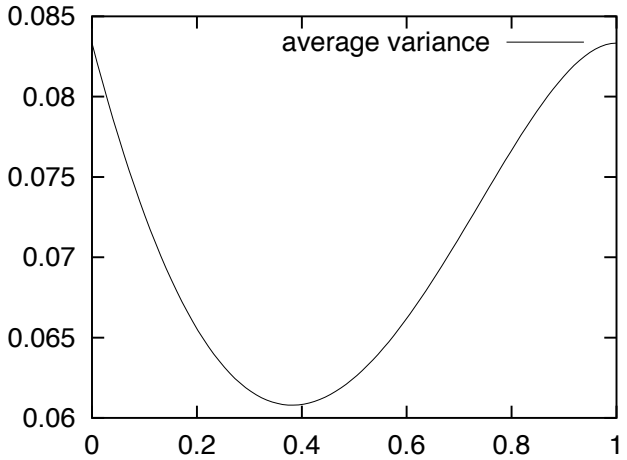


Mauve: motivating example

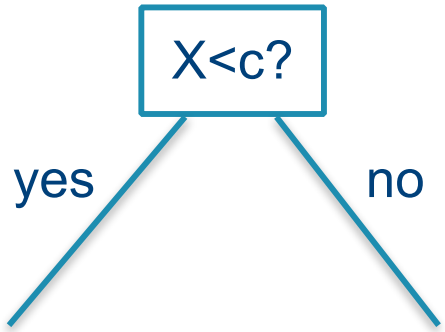
Minimization of average variance does not yield a good split



(a)



(b)

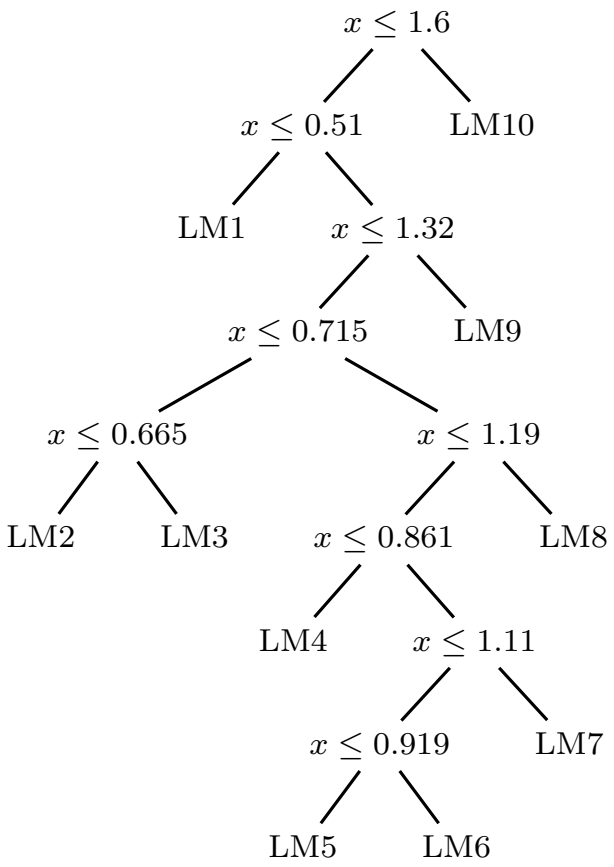


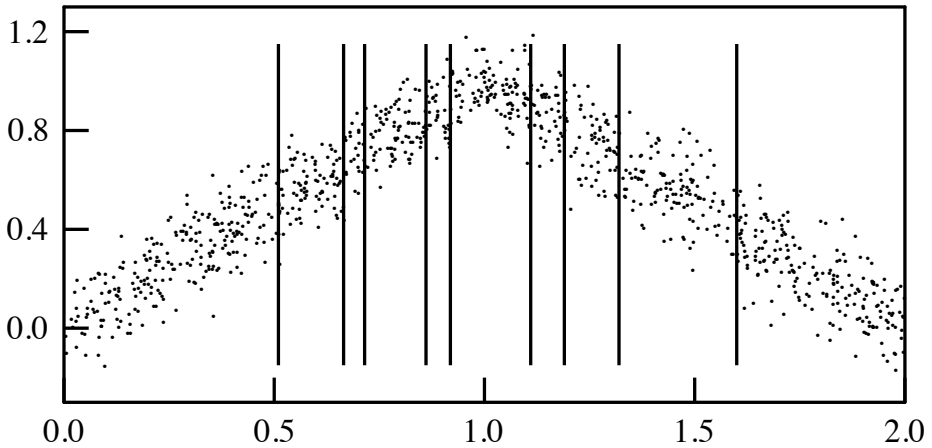
Average variance is not reduced for c=1 (optimal test), and is maximally reduced for c ≈ 0.4

KU LEUVEN

Mauve: motivating example

As a result, M5 builds a complex tree on such data



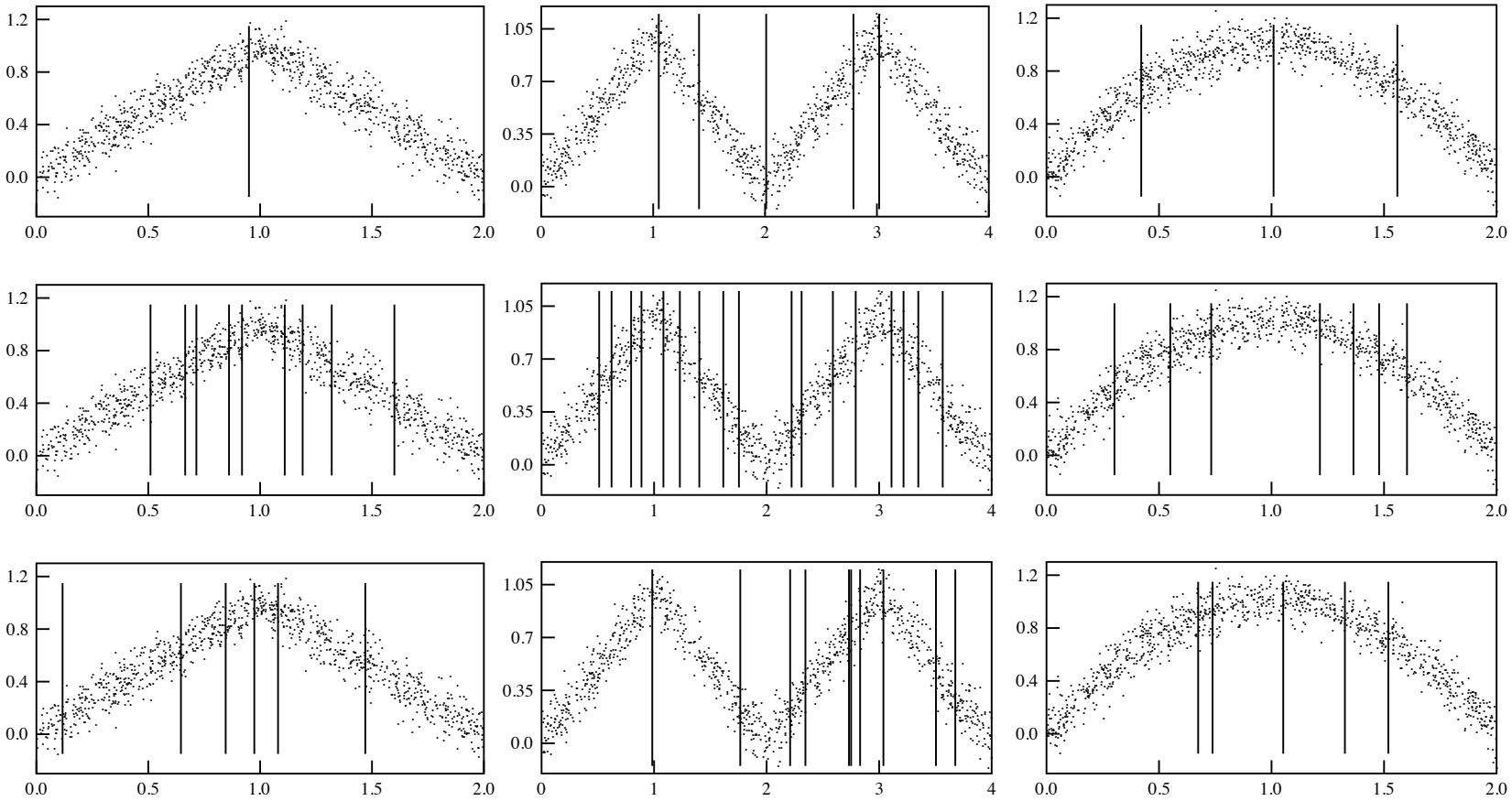


LM1: y = 0.003 + 1.010x
LM2: y = 0.577
LM3: y = 0.713
LM4: y = 0.470 + 0.411x
LM5: y = 2.163 - 1.449x
LM6: y = 1.260 - 0.296x
LM7: y = 0.859
LM8: y = 0.730
LM9: y = 1.630 - 0.744x
LM10: y = 1.806 - 0.895x

KU LEUVEN

Mauve vs. M5’

Experimental comparison on synthetic data



Mauve

M5’

Random splits



Multi-target trees

Predicting multiple variables with one tree

KU LEUVEN

Multi-target trees

Classification and regression trees are very popular for predicting one target variable

But the principle of variance reduction is easily generalized to predicting vectors !

This allows us to predict multiple variables at the same time, using one tree

Question: will that work well?

KU LEUVEN

Multi-label classification

Multi-label classification : predict for each instance a *set* of labels

E.g., assigning keywords to texts:

- X = description of the text (e.g., bag-of-words)
- Y = a set of keywords (e.g., {animals, climate change, North Pole})

How can we learn a model that predicts a set of labels?

How can we predict the sets?

KU LEUVEN

Multi-label classification

Two straightforward approaches are often mentioned:

Option 1: ("binary relevance")

- Learn one binary (yes/no) decision tree for each label
- TreeA predicts a (yes/no), TreeB predicts b, etc.

Option 2: ("label powersets")

- Consider each set as a separate label
- For 5 original labels a,b,c,d,e, we get 32 combined labels: -, a, b, c, d, e, ab, ac, ad, ae, bc, …, abc, …
- Generally, from n labels, we get 2^n combined labels
- Learn a tree that predicts the combined label

Multi-label classification

A third approach, very simple to implement with decision trees:

Option 3: (vector encoding)

- encode each set as a 0/1 vector, e.g. {a,b,d} = [1,1,0,1,0]
- use a learner that can learn models that predict vectors

Very straightforward for decision trees: requires almost no changes in the algorithm

- best_test: variance of Y becomes variance of vectors
- info: store the mean vector of all instances in that leaf (= vector of proportions)

E.g., leaf contains { [1,1,0,1,0], [1,1,0,0,0], [1,1,0,1,1] } → [1, 1, 0, 0.67, 0.33]

KU LEUVEN

Multi-label trees: prediction

- To make a prediction for a new instance:
  - find the leaf it belongs to
  - predict all labels for which the proportion in the label is above some given threshold (often 0.5, but could be other)

- E.g., [1, 1, 0, 0.67, 0.33] → {a,b,d}

KU LEUVEN

Hierarchical multilabel classification

Similar to the previous problem, but now the labels are organized into a hierarchy.

A label can only occur in a set if all its ancestors also occur

Example applications:

- Classifying texts into newsgroups
- Assigning functions to proteins

KU LEUVEN

Hierarchical multilabel classification

With decision trees: we can do *exactly* the same as regular multilabel classification (vector encoding)

Constraint “a label can only occur in an instance’s label set if all its ancestors also occur” is automatically fulfilled

We can assign weights to different labels, to punish high-level mistakes more strongly than low-level mistakes

KU LEUVEN

An application & empirical validation in functional genomics

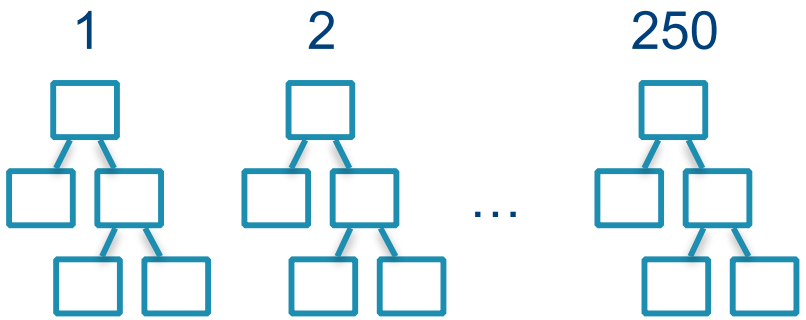
- Predict the biological functions of a protein based on a description of it
- 250 functions are distinguished, organized hierarchically
- A single protein may have multiple functions in intracellular processes
- So this is an HMC problem

Blockeel et al. (2006), **Decision Trees for Hierarchical Multilabel Classification: A Case Study in Functional Genomics**. In Proc. of PKDD 2006: 18-29.

KU LEUVEN

“Normal” classification trees

- Advantage of trees: fast and interpretable
- Learn a separate tree for each function? 250 functions = 250 trees: not so fast and interpretable anymore!



Descriptive attributes | Labels

Name	A1	A2	A3 …..	An	1	2	3	4	5	…	249	250
G1	…	…	…	…	x			x	x		x	
G2	…	…	…	…		x		x			x	
G3	…	…	…	…	x				x			x
…	…	…	…	…	…	…	…	…	…	…	…	…

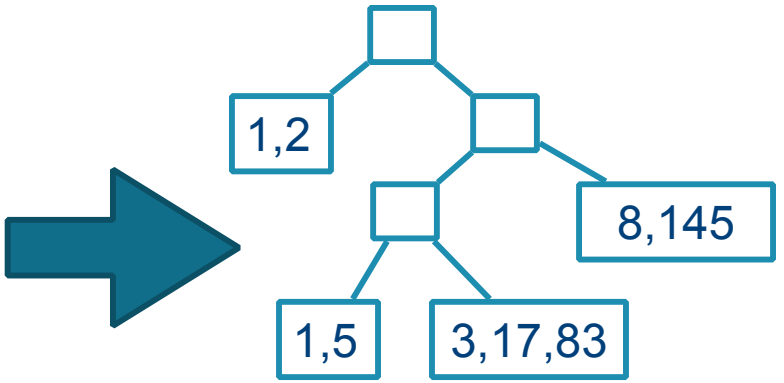
KU LEUVEN

HMC trees

- Learn 1 tree that predicts a 250-dimensional class vector
- “Variance” (for heuristics) = mean squared distance to mean in 250-D space

Descriptive attributes | Labels

Name	A1	A2	A3 …..	An	1	2	3	4	5	…	249	250
G1	…	…	…	…	x			x	x		x	
G2	…	…	…	…		x		x			x	
G3	…	…	…	…	x				x			x
…	…	…	…	…	…	…	…	…	…	…	…	…



Good interpretability

Converting trees to if-then rule format gives short and interpretable rules

IF Nitrogen_Depletion_8_h <= -2.74 **AND**
Nitrogen_Depletion_2_h > -1.94 **AND**
1point5_mM_diamide_5_min > -0.03 **AND**
1M_sorbitol___45_min_ > -0.36 **AND**
37C_to_25C_shock___60_min > 1.28
THEN 40, 40/3, 5, 5/1

For class 40/3: Recall = 0.15; precision = 0.97.
(rule covers 15% of all class 40/3 cases, and
97% of the cases fulfilling these conditions are indeed 40/3)

Decision trees in practice



KU LEUVEN

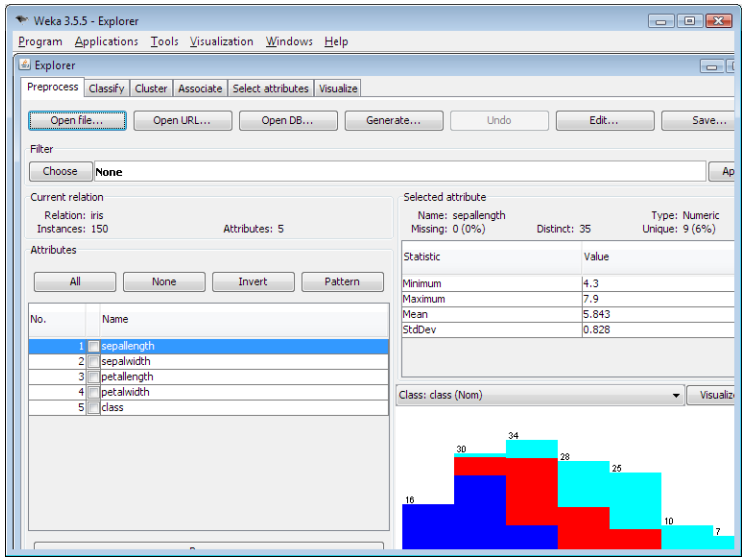
Weka

A popular, easy-to-use, free data mining tool

Available for download at http://www.cs.waikato.ac.nz/ml/weka/

Includes a highly optimized decision tree learner J48 and model tree learner M5' (variants of C4.5 and M5, respectively)





KU LEUVEN

Python: scikit-learn

- Python has become increasingly popular among data scientists
- Python package **scikit-learn** is very popular for machine learning; includes (basic!) decision tree support

```
>>> from sklearn.datasets import load_iris
>>> from sklearn import tree
>>> iris = load_iris()
>>> clf = tree.DecisionTreeClassifier()
>>> clf = clf.fit(iris.data, iris.target)

>>> from matplotlib import pyplot as plt
>>> fig = plt.figure(figsize=(25,20))
>>> _ = tree.plot_tree(clf,
                       feature_names=iris.feature_names,
                       class_names=iris.target_names,
                       filled=True)
>>> fig.show()
```

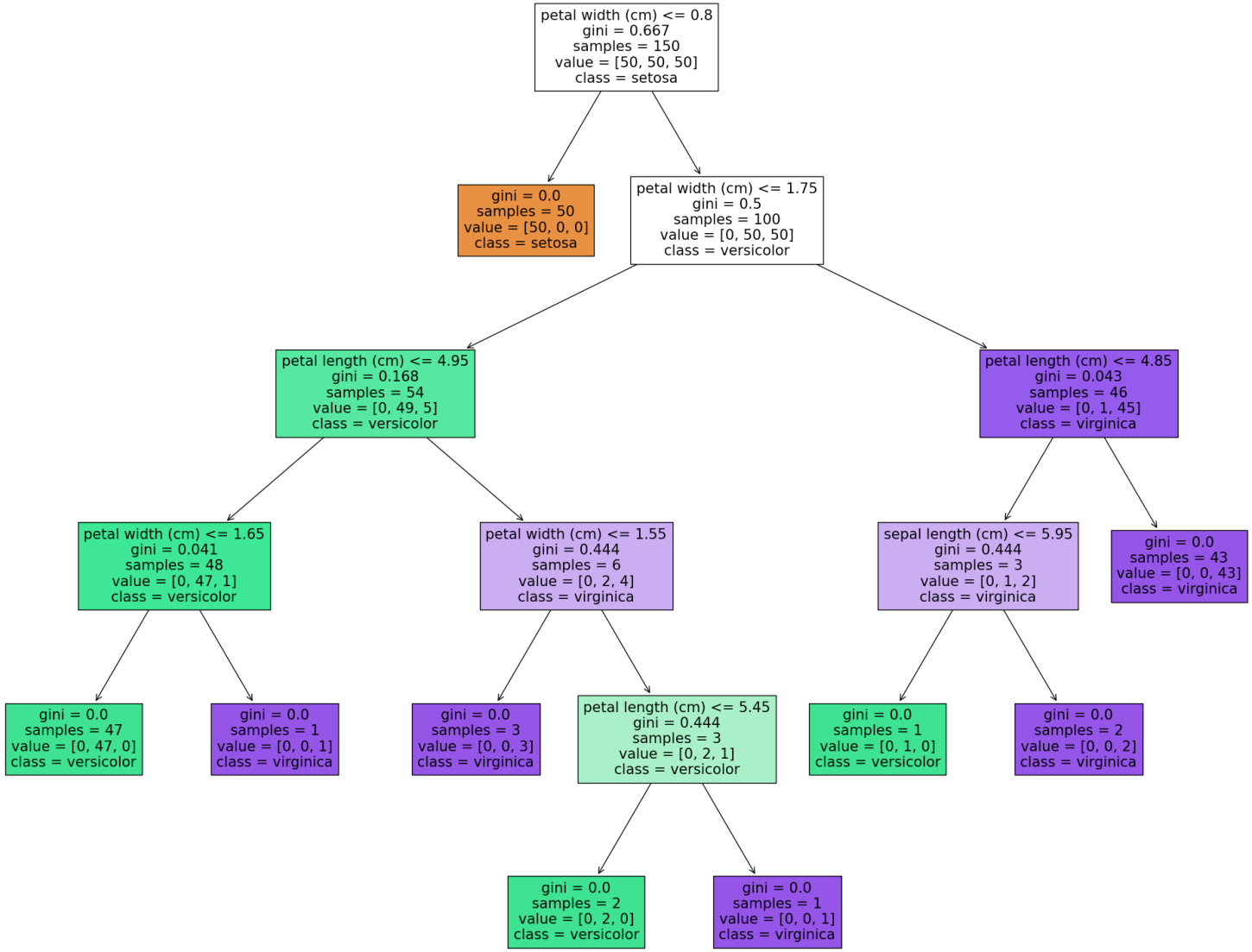
Load data & learn a decision tree from it

Plot the tree

Example by Piotr Płoński, https://mljar.com/blog/visualize-decision-tree/

KU LEUVEN

Python: scikit-learn



KU LEUVEN

Python: scikit-learn

Some methods defined for DecisionTreeClassifier objects:

`apply`(X[, check_input])	Return the index of the leaf that each sample is predicted as.
`cost_complexity_pruning_path`(X, y[, ...])	Compute the pruning path during Minimal Cost-Complexity Pruning.
`decision_path`(X[, check_input])	Return the decision path in the tree.
`fit`(X, y[, sample_weight, check_input, ...])	Build a decision tree classifier from the training set (X, y).
`get_depth`()	Return the depth of the decision tree.
`get_n_leaves`()	Return the number of leaves of the decision tree.
`get_params`([deep])	Get parameters for this estimator.
`predict`(X[, check_input])	Predict class or regression value for X.
`predict_log_proba`(X)	Predict class log-probabilities of the input samples X.
`predict_proba`(X[, check_input])	Predict class probabilities of the input samples X.
`score`(X, y[, sample_weight])	Return the mean accuracy on the given test data and labels.
`set_params`(**params)	Set the parameters of this estimator.

Other software

Many other decision tree learners exist, either as stand-alone software, or integrated in data analysis software (R, SAS, SPSS, …)

Almost all statistical packages, these days, offer decision trees as an analysis method